Sri Sivasubramaniya Nadar College of Engineering

Department of Computer Science and Engineering

Title

PAYLOAD-AWARE TWO-TIER UAV ROUTE PLANNING FOR LAST-MILE DELIVERY

REVIEW REPORT – II

Submitted by

Thirumurugan RA (3122235001149)
Meghna Manimaran (3122235001079)
Shrika SP Thota (3122235001125)

Under the guidance of

Dr. N. Sujaudeen, Assistant Professor

Academic Year: 2026–2027

Contents

Abstract **1**

1 Introduction **2**

1.1 Background 2

1.2 Motivation 4

1.3 Related Work 4

1.4 Research Gap 6

1.5 Objectives 8

1.6 Scope 8

2 First Review – Queries and Answers **9**

2.1 Query 1: The Complexity of the Work Is Not High Enough 9

2.1.1 Better node selection for the airspace graph 9

2.1.2 Flight risk estimation from wind speed and direction 10

2.1.3 What the new graph forced on the rest of the system 11

2.2 Query 2: Phase I Running Until December Is Not Justified 13

2.2.1 Benchmarking and evaluation is a repeated cycle 13

2.2.2 Bringing the simulation closer to real conditions 15

2.2.3 Revised Phase I plan 16

3 Problem Statement **16**

3.1 Weather Dependence of the Problem 17

3.2 Inputs, Outputs and Success Criteria 17

4 Methodology **18**

4.1 Architectural Design for Proposed System 19

4.2 Modules 21

4.2.1 Module 1: Airspace Graph Builder 22

4.2.2 Module 2: Tier 1 Pareto Front Engine (NAMOA*) 29

4.2.3 Module 3: Baseline Planners 33

4.2.4 Module 4: Tier 2 Payload Metadata Interpreter 34

4.2.5 Module 5: Evaluation and Benchmarking 38

4.2.6 Module 6: Mission Export and Ground-Station Dashboard . . 40
4.2.7 Module 7: Simulation and SITL Validation 40

5 Implementation **42**
5.1 Development Environment and Tools 43
5.2 Code Organisation 44
5.3 Module 1: Airspace Graph Builder 45
5.3.1 Weather and terrain data 45
5.3.2 OSM layers and spatial indexes 45
5.3.3 Lattice builder 45
5.3.4 Main parameters 47
5.3.5 Entry points 47
5.4 Module 2: Tier 1 Engine and Front Store 47
5.4.1 Label table 47
5.4.2 Diagnostics 48
5.4.3 Front store 48
5.5 Module 3: Baselines 48
5.6 Module 4: Tier 2 Selector 48
5.7 Module 5: Evaluation 49
5.8 Module 6: Mission Export and Dashboard 49
5.9 Module 7: Simulation 50
5.10 Testing 50

6 Results and Discussion **51**
6.1 Experimental Setup 52
6.2 Tier 1 Front Quality Against the Baselines 52
6.3 Effect of Wind Speed and Direction 54
6.4 Contribution of Each Risk Layer 56
6.5 Tier 2 Selection 58
6.6 Simulation Results So Far 60
6.7 Comparison With Existing Work 61
6.8 Discussion and Next Steps 61

List of Figures

2.1 The benchmark, evaluate and improve cycle planned for October to December 14

2.2 Revised project timeline, July 2026 to March 2027 16

4.1 System architecture 20

4.2 Mission sequence from payload tag entry to the flown route 21

4.3 Module 1 pipeline: data acquisition, node selection, edge construction and the shared cost pipeline 23

4.4 Selection rule applied to each candidate lattice node 25

4.5 Per-edge wind decomposition into energy and risk 28

4.6 Tier 1 activity flow: NAMOA* with a label table 32

4.7 Front store decision: reuse the stored front or rebuild it 33

4.8 Tier 2 activity flow with validation, feasibility and weather hold . . . 38

4.9 Gazebo and ArduPilot SITL simulation pipeline 41

5.1 Core classes of the planner 44

6.1 Hypervolume per method and scenario (first pass) 54

6.2 Runtime per method and scenario (first pass) 54

6.3 Front size, minimum energy and minimum risk on S2 as a 25 km/h wind turns through 360°; the dashed line is calm air 56

6.4 Edge risk maps for the ablation variants (fixture layers) 58

6.5 Tier 2 routes for the four presets on a 2 km Thiruvanmiyur lattice . . 60

List of Tables

1.1 Summary of related work 5

2.1 System as presented at Review I compared with the system at Review II 12

3.1 Inputs, outputs, constraints and success criteria 18

4.1 Risk components (every $\rho_k \in [0, 1]$) and their weights 26
4.2 Semantic mapping: payload metadata tags to Tier 1 routing objectives 35
4.3 Reference payload presets and their derived weights 36

5.1 Languages, libraries and tools 43
5.2 Source files and the modules they implement 44
5.3 Main Module 1 parameters (`config.py`) 47
5.4 Test suites and what they verify 51

6.1 Scenarios used in this chapter 52
6.2 First-pass benchmark on synthetic scenarios (HV higher is better; IGD and Δ lower are better) 53
6.3 Effect of wind direction on the S2 front (25 km/h wind, 35 km/h gust) 55
6.4 Directional edge costs on a flat lattice, 25 km/h wind from 90° 56
6.5 Ablation on the 6 km Thiruvanmiyur lattice (fixture layers, 30 km/h wind from 225°) 57
6.6 Tier 2 picks on the S2 (16 routes) and S3 (115 routes) fronts; last column is latency in ms 59
6.7 Comparison with the reviewed works 61

Abstract

In UAV systems for medical and emergency last-mile delivery, conflicting objectives must be balanced: higher flight risk is carried by shorter routes, more battery is drained by low-risk corridors, and the fastest paths are rarely achieved by energy-efficient routing. No single fixed trade-off serves all payload types, yet the same objective weights are applied by existing planners to every mission regardless of the payload. In existing multi-objective planners, including entropy-based, diversity-driven, and crowding-distance Pareto selection methods, objective weights are encoded as a fixed scalar before path search begins. No influence is exerted by payload properties such as fragility class, payload weight class, and delivery urgency over which Pareto-optimal route is chosen. When payloads change, weights must be manually re-tuned and planners re-run by operators, which is unworkable under field conditions. The proposed work is a Payload-Aware Two-Tier UAV Route Planning system, wherein Pareto front construction is separated from route selection and an explicit algebraic bridge is added between Tier 1 routing objectives and Tier 2 payload metadata. In Tier 1, a multi-objective A*/Dijkstra graph-search is executed over an OpenStreetMap airspace graph, producing a Pareto-optimal route set across distance, energy consumption, and flight risk. In Tier 2, four operator-assigned payload tags are read and objective weights derived: w_{risk} is scaled by fragility class, w_{distance} by delivery urgency, and w_{energy} by payload weight class. Every Pareto point is scored by these weights via a utility function, and the argmax is selected as the mission route. Evaluation will be conducted on synthetic graphs (50, 200, 500 nodes) against single-objective A* and MOPSO baselines using hypervolume, spread, and Tier 2 accuracy. Hardware validation will use a Pixhawk quadcopter via ArduPilot SITL simulation and outdoor flights. Planned experiments will quantify Tier 1 gains over both baselines, evaluating whether non-extreme Pareto points are selected for mixed-priority payloads. The principal contribution is a formula-driven mapping from metadata to routing weights, linking mission context directly to Pareto selection without expert intervention. Route choice is set by payload properties rather than context-blind diversity criteria. The Pareto front is rebuilt periodically, serving all missions between updates.

Keywords: UAV route planning; multi-objective optimisation; NAMOA*; Pareto front; payload-aware selection; wind-aware risk; last-mile delivery

1. Introduction

Unmanned aerial vehicles (UAVs) are a useful platform for last-mile delivery in scenarios where ground transport is impractical: medical supply chains to remote clinics, emergency drug drops during disaster response, and fragile diagnostic sample transport where road vibration risks contamination. In each scenario the UAV must fly a route from a dispatch hub to a delivery point across an airspace that contains obstacles, regulated no-fly zones, and variable wind corridors. A good route balances three conflicting objectives: total flight distance, battery energy use, and exposure to high-risk segments.

This three-way conflict is exactly what multi-objective UAV path planning is built to handle. Rather than collapsing distance, energy, and risk into one score before the search runs, a multi-objective planner keeps all three objectives separate through the search and returns a Pareto-optimal front: a set of routes where none can be improved on one objective without giving up ground on another [1]. The front tells the operator what trade-offs are available; it does not, by itself, say which route to fly.

One common way to sidestep the trade-off question is to form a weighted sum of the three objectives into a single scalar cost function and then run a conventional path planner such as A* on it. This scalar collapses the multi-objective problem into a single-objective one, and the operator adjusts the three weights to express the mission's priority before each flight. The approach works for a fixed mission type, but it fails immediately when the payload changes: emergency medicine requires a completely different weight profile from fragile biological samples or disaster-relief bulk supplies [2]. Manually re-tuning and re-running the planner between missions is operationally impractical in time-critical field conditions.

1.1 Background

Building the Pareto front, as introduced in Chapter 1, is well studied: the field has produced graph-search and population-based methods that generate well-spread, non-dominated route sets efficiently. The question that decides whether any such system is useful in practice is how the final route gets picked from the front. Section 1.3 reviews three directly relevant papers and finds the same answer in each: delivery context plays no role. None of them let what the UAV is carrying influence route choice, or even ask which route should be chosen for a given payload. That is the gap this project addresses.

Autonomous UAV systems for medical and emergency last-mile delivery work

under a constraint that separates them from general routing problems: the best route trade-off changes with the payload. A fragile diagnostic sample needs a low-vibration, low-turbulence corridor even at the cost of a much longer flight time. Emergency insulin needs the shortest path regardless of risk. Bulk disaster-relief supplies need the most energy-efficient route to maximise delivery range.

Payload-awareness itself is not absent from the UAV literature: variable payload mass has been used to adapt the flight-speed and energy-consumption *model itself* during trajectory planning, recomputing the underlying physical equations as mass changes across a multi-drop mission [3]. What is absent is any use of payload metadata to *select* a point on a pre-computed multi-objective Pareto front. Tier 2's payload weight tag is a discrete, ordinal class (light/medium/heavy) that is one of four metadata tags algebraically blended into a static selection weight once per mission; it never alters the per-edge energy model or triggers re-planning, unlike [3]'s continuous mass-driven physics adaptation. Tier 2 closes that gap by selecting among a fixed front from metadata, rather than adapting the model to continuous mass.

Real-world deployments make the operational cost of payload-blind routing concrete. In January and May 2019, Swiss Post–Matternet M2 drones carrying clinical laboratory specimens on fixed, payload-invariant routes crashed in Zürich; post-incident analysis confirmed that the aircraft flew identical corridors regardless of whether the cargo was a routine parcel or an irreplaceable blood specimen, with no routing adjustment for payload fragility [4]. Independent laboratory work explains why this matters: controlled studies on drone-transported medicines have shown that the vibration and turbulence encountered along UAV flight paths cause measurable payload degradation—insulin exhibits fibrillation risk, biologics undergo protein denaturation, and blood samples show elevated haemolysis indices when the route includes high-vibration segments [5, 6]. A comparative UAV blood-transport study confirmed that pre-analytic integrity varies significantly with flight-path characteristics, with routes of greater altitude variation and turbulence producing higher rates of sample rejection at the receiving laboratory [7]. In every case the root cause is the same: the route planner had no knowledge of what was being carried, so it could not prefer smoother, lower-risk corridors for fragile payloads or shortest paths for time-critical medicine. Operational deployments such as Zipline's blood-delivery network in Rwanda—which serves remote clinics up to 80 km away on a single battery charge—further illustrate the energy dimension of the gap: heavier blood-product payloads draw more hover and lift power and shorten effective range, yet the system plans the same fixed route regardless of payload weight, with no mechanism to select an energy-efficient detour for heavy cargo or a direct short route when an urgent, lighter payload makes that trade-off optimal [8].

1.2 Motivation

Three real operational scenarios motivate this work, and their routing requirements do not agree.

Fragile medical samples such as tissue biopsies and blood specimens are sensitive to vibration and to temperature. The routing priority is to minimise flight risk, taken here as time spent in turbulent corridors, sudden altitude changes, and wind-exposed segments. A route that adds 40% distance but halves the chance of vibration damage is the better one.

Emergency medicine such as insulin or antivenom has a strict time window. The priority is the shortest route, so transit time is lowest; flight risk and energy come second. For a 15-minute delivery window, the shortest path wins outright.

Disaster-relief bulk supplies such as water purification tablets and ration packs are heavier than medical payloads and are neither urgent nor fragile. The priority is the lowest battery drain, so a single charge covers as many deliveries as possible.

No existing UAV planner switches between these three routing strategies from a payload tag. The proposed two-tier system does this with no expert intervention and no replanning. In the UAV delivery literature, payload metadata (fragility class, payload weight class, delivery urgency) has never been the sole input to automatic Pareto point selection. That is the gap this project targets.

1.3 Related Work

Multi-objective UAV path planning draws on both graph-search and evolutionary optimisation [1]. Within it, route selection from a computed Pareto front is usually guided by the crowding-distance rule introduced by NSGA-II [9], or by an entropy criterion that keeps the archive diverse while a manually weighted scalar sum makes the final pick; neither considers delivery context. Payload information, where it appears at all, is limited to continuous mass folded directly into the energy/speed model itself, not discrete metadata used to select among a pre-computed Pareto front. Three papers most directly relevant to the proposed work are reviewed below; Table 1.1 gives a structured comparison.

Table 1.1: Summary of related work

S.No.	Author/Year	Proposed Method	Key Findings	Limitations
1	Chokkalingam & Ganesan (2026) [2]	Pareto-optimal particle swarm optimisation (POPSO) with dynamic parameter adaptation and Pareto–entropy archive selection to maintain front diversity during optimisation; the final single route is then read off via a weighted-sum scalarisation whose weights are set empirically per mission.	Entropy-guided archiving improves front diversity and adaptability to real-time environmental changes over static-weight baselines.	Final route choice still reduces to a manually re-tuned weighted sum per mission; payload metadata never incorporated, so fragile samples and emergency medicine receive the same hand-tuned selection procedure from the same front.

continued on next page

Table 1.1 (continued)

S.No.	Author/Year	Proposed Method	Key Findings	Limitations
2	Tsoutsouras et al. (2020) [3]	Adapts flight speed and the energy-consumption model to the UAV's instantaneous payload mass during trajectory planning for multi-user delivery missions, updating the trajectory as mass changes between drop-offs.	Mass-aware trajectory adaptation reduces energy consumption relative to mass-agnostic planning across multi-stop delivery routes.	Uses continuous payload mass only as a physical parameter inside the energy/speed model itself; builds no multi-objective Pareto front and has no notion of discrete payload metadata (fragility class, payload-weight class, urgency) selecting among pre-computed routing trade-offs.
3	Ren et al. (2019) [10]	Crowding-distance-based NSGA-II (CDNSGA-II) for multi-objective UAV path planning, balancing distance and safety over an octree-represented urban environment.	CDNSGA-II finds well-spread, collision-free Pareto-optimal paths efficiently in octree-modelled urban airspace.	Crowding distance encodes only the geometric spread of the front; it cannot represent urgency, fragility, or payload weight, and mission context plays no role in selection.

1.4 Research Gap

Two of the three reviewed papers treat Pareto-front diversity as the central design concern, and only one of those two uses it to *select* a route: the crowding distance of [10] picks a single point directly from the front, while [2] uses entropy only to keep the

archive *diverse* during optimisation and falls back to a manually weighted scalar sum, re-tuned per mission by a human operator, to make the actual final pick. [3] does not build a Pareto front at all: it folds continuous payload mass into the energy model's equations of motion and adapts flight speed as mass changes across a multi-drop mission, but never selects among competing routing trade-offs by payload. Whether the mechanism is geometric diversity or continuous mass-driven physics adaptation, none of the three use payload weight, or any other payload property, as a discrete input to a Pareto-point *selection* mechanism; in [3], mass reshapes the cost model itself and never enters a selection formula. This is the gap the proposed system closes: Tier 2 treats payload weight as one of four ordinal metadata tags algebraically blended into a selection weight, and does not use it as a continuous physical parameter inside the cost model.

The gap is not an implementation detail; it is a design assumption shared by all three approaches. Entropy and crowding distance are both defined purely over the geometry of the Pareto front; mass-aware trajectory adaptation is defined purely over the physical energy model, with mass entering as a continuous parameter to that model, never as a selection criterion. None of these mechanisms has an interface through which discrete payload metadata, as opposed to continuous mass or point geometry, can enter a *selection* step. Adding payload information to them would mean replacing the selection criterion outright, which is what the proposed Tier 2 utility scoring does.

This work addresses three specific gaps.

Gap 1. Existing UAV multi-objective planners offer no payload-aware route selection: they apply a fixed geometric criterion independent of payload type, or no selection mechanism at all. Tier 2 maps four payload tags directly to Pareto point selection through a formula.

Gap 2. Existing planners do not re-use one front across mission types; they re-run the full multi-objective search whenever the mission context changes. The two-tier separation decouples front construction from mission arrivals: Tier 1 refreshes the front on a periodic cadence, independent of payload, and Tier 2 selects a route for any new payload tag against the current front with no re-run.

Gap 3. No algebraic bridge links a payload description to the routing trade-off it implies (e.g. fragility $\rightarrow$ minimise risk, urgency $\rightarrow$ minimise distance); the mapping has never been formalised. The weight-derivation formula supplies that bridge, and the mapping it defines is deterministic and reproducible.

1.5 Objectives

1. To formulate multi-objective UAV route planning as a Pareto front precomputation task over a delivery airspace graph, and to distinguish the deterministic graph-search approach from stochastic population-based alternatives on theoretical grounds.

2. To design a payload metadata schema and a rule-based mapping that turns mission context (fragility class, delivery urgency, payload weight class) into Pareto point selection weights with no expert weight tuning.

3. To benchmark Tier 1 route quality and Pareto front coverage against single-objective A* and MOPSO baselines on identical delivery scenarios, using hypervolume, Pareto spread, and Inverted Generational Distance (IGD) as quality indicators.

4. To quantify Tier 2 selection accuracy by measuring how closely the selected route's characteristics match the payload tag requirements across mission types (fragile medical sample, emergency medicine, disaster-relief supplies).

5. To establish the system's novelty: payload metadata is the sole input to Pareto point selection, in place of context-blind entropy and diversity criteria, through a deterministic algebraic mapping from payload description to route choice.

1.6 Scope

This project covers two phases. Phase I delivers a complete, validated software simulation system including the multi-objective A* graph-search engine (Tier 1), the payload-driven route selector (Tier 2), a MOPSO baseline, and a benchmarking suite evaluated on programmatically generated airspace graphs at three scales (50, 200, and 500 nodes). Phase II extends the validated software to physical hardware: a Pixhawk-equipped F450 quadcopter with a Raspberry Pi 5 companion computer, validated through ArduPilot SITL simulation and outdoor flight tests. The Tier 1 Pareto front is treated as valid for a bounded refresh window: it is rebuilt on a periodic cadence, or when environmental drift (e.g. a substantial wind-condition shift) exceeds a defined threshold. Missions that fall inside that window are served by Tier 2 against the current front, with no per-mission recompute. The project does not address multi-UAV coordination, dynamic obstacle avoidance during flight, or real-time path replanning.

2. First Review – Queries and Answers

The panel raised one main concern at Review I, and it had two parts. The first part was that the technical complexity of the work looked too low. The second part followed from the first: if the system is that simple, a Phase I that runs until December is hard to justify. We agree that the version shown at Review I invited both comments. This chapter answers each part in turn. Chapters 3.2 to 5.4 carry the full detail behind the answers.

2.1 Query 1: The Complexity of the Work Is Not High Enough

Query. The panel observed that the overall complexity of the proposed system was low for a final-year project.

Answer. For the Review I prototype this was a fair reading. The airspace graph was the OpenStreetMap road network fetched with `osmnx`, so the drone could only fly along streets. Energy was distance multiplied by a random altitude factor. Risk came from two hand-placed obstacle circles plus noise, and wind was one number that added the same penalty to every edge. With costs like these the three objectives hardly conflicted, the Pareto front often held one or two routes, and Tier 2 had very little to choose from.

Since Review I we have rebuilt the part of the system that every later stage depends on: the airspace graph and the cost model on its edges. Two changes account for most of the added complexity. The node choosing for the airspace graph is now a multi-source selection rule instead of a road lookup, and flight risk is now estimated from wind speed *and* wind direction for every edge separately. Both are described below, followed by the knock-on changes they forced in Tier 1, Tier 2 and the tooling around them.

2.1.1 Better node selection for the airspace graph

The road graph has been replaced by a three-dimensional airspace lattice. A UAV is not bound to roads, so the candidate waypoints are laid out as a regular grid (200 m spacing over a 6 km box around Thiruvanmiyur, Chennai, which gives 900 candidates). A candidate does not enter the graph automatically. It has to pass a selection rule that combines four data sources:

(i) Ground elevation from a digital elevation model. The value comes from a chain of four providers (a local raster, the OpenTopography Copernicus 30 m service, the Open-Meteo elevation service [11], and a flat fallback), each cached on disk.

(ii) The tallest building inside the node's 200 m cell. Footprints come from OpenStreetMap and are held in a Shapely STRtree spatial index. A building's height is read from its `height` tag, else from `building:levels` $\times$ 3 m, else from a per-type default.

(iii) The legal ceiling. The node's cruise height above ground is $\max(60\,\text{m},\ h_{\text{bldg}} + 15\,\text{m})$. If that exceeds the 120 m limit of the Drone Rules, 2021 [12], the node is blocked.

(iv) Restricted airspace. Aerodromes, military land, prisons, protected areas and government sites are fetched as polygons. A node inside one is removed.

Edges get the same treatment. Each surviving node is linked to its eight neighbours in both directions. An edge whose straight segment cuts a restricted polygon is dropped, an optional clearance filter (used by the simulator) drops edges that pass less than 15 m over a buffered roof or over the terrain, and finally only the largest strongly connected component is kept so that every depot and destination pair is reachable.

Every edge that survives is then sampled every 25 m along its length, and seven static terms are computed from those samples: building proximity scaled by vertical margin, terrain clearance and grade, population exposure (a SORA-style class lookup [13]), soft distance to restricted zones, fraction over water, urban-canyon communication loss, and building density. For the default region this is roughly 6,800 directed edges, about 75,000 sample points and around 375,000 spatial-index queries per build. At Review I, building the graph was a single library call.

The result is a graph whose nodes carry a real altitude above mean sea level, whose edges have a real grade and a three-dimensional length, and whose risk changes across the map with buildings, crowds, water and restricted zones.

2.1.2 Flight risk estimation from wind speed and direction

At Review I, wind was a single scalar applied to every edge, so it only penalised the number of hops. The cost model now resolves the wind against the heading of each edge. Wind is read from the Open-Meteo forecast at 80 m, which is close to the 60–120 m cruise band. When only the 10 m value is available it is lifted to cruise height with the power law $v_h = v_{10}(h/10)^{0.25}$ [14]. The edge heading comes from the forward-azimuth

formula, and the wind is split into a headwind component $HW = v\cos(\theta_{\text{edge}} - \theta_{\text{from}})$ and a crosswind component $CW = v\,|\sin(\theta_{\text{edge}} - \theta_{\text{from}})|$.

These two components feed both objectives. Headwind and crosswind raise the energy of an edge, while a tailwind is given no credit so that energy never drops below distance and the Tier 1 heuristic stays admissible. Crosswind drives a risk term, the excess of gust over mean wind drives another, and wind multiplied by local building density gives an urban turbulence term. Together with rain and visibility these sit inside an 11-component weighted risk score, and a separate weather gate stops all flights during thunderstorms, heavy rain, low visibility or gusts above the airframe limit.

The first version of this wind model had six defects that we found and fixed while implementing it, including an inverted headwind sign, a 5.5-hour time-zone offset when reading the hourly forecast, and a bearing formula without the $\cos\varphi$ correction. That says something about how much care the directional model needs.

The effect on the graph is large. Edge costs are now genuinely directional: flying $u \to v$ and $v \to u$ cost different amounts. Under a 25 km/h wind from the east, east-bound lattice edges need 27.8% more energy per kilometre than west-bound edges of the same length, and north–south edges carry six times the crosswind risk of east–west edges (0.225 against 0.038). On the 196-node benchmark graph, turning a 25 km/h wind around the compass in 15° steps, with nothing else changed, moves the cheapest route's energy between 24.6 and 31.6 units (a 28.5% swing) and changes the Pareto front from 36 routes in calm air to between 6 and 15 routes. The route chosen for a fragile medical payload changes with the wind as well. Section 6.3 gives the full numbers.

2.1.3 What the new graph forced on the rest of the system

A richer graph made the downstream stages harder, and each had to be reworked.

- Tier 1 was rewritten as NAMOA* [15] with a label table. It keeps one label per non-dominated partial path, uses a vector heuristic $[h, h, 0]$ that we prove admissible for the new cost model, prunes on $g + h$ against the current front, and reports when its expansion cap truncated the search. On the 500-node dense scenario the exact front has 115 routes.

- The front now depends on the weather, so reusing it blindly is unsafe. A front store keeps each front with a SHA-256 hash of the static graph, the weather it was built for and a 3-hour validity window, and rebuilds it when the wind speed moves by more than 10 km/h, turns by more than 45°, gusts change by more

than 15 km/h or the go/no-go state flips.

- Tier 2 gained input validation, a battery feasibility filter that converts each route's energy into watt-hours for the payload's mass class, a weather HOLD, and a structured selection record.
- A Gazebo and ArduPilot SITL pipeline now flies the chosen route through a three-dimensional model of the delivery area and measures how closely it was followed.

Table 2.1 sets the two versions side by side.

Table 2.1: System as presented at Review I compared with the system at Review II

Aspect	**Review I**	**Review II**
Airspace graph	OSM road network via `osmnx`; drone tied to streets	3-D lattice (200 m, 6 km box) with a per-node selection rule and per-edge pruning
Node altitude	Not modelled	Ground (DEM) $+ \max(60\,\text{m}, h_{\text{bldg}} + 15\,\text{m})$; nodes above 120 m AGL blocked
Data sources	OSM roads only	Open-Meteo weather, four-provider DEM chain, OSM buildings, restricted zones, water and land use through a multi-endpoint Overpass client, all cached
Energy objective	Distance × random altitude factor	3-D distance with grade, headwind and crosswind terms
Risk objective	Two synthetic circles plus noise	11 weighted components; wind, gust and turbulence depend on edge heading; cumulative exposure
Edge costs	Symmetric	Directional ($u \to v \neq v \to u$ under wind)
Weather handling	None	Go/no-go gate (thunderstorm, rain, visibility, gust) respected by Tier 2
Tier 1	MO-A* with a visited set and a path copy per push	NAMOA* label table, admissible vector heuristic, $g + h$ pruning, truncation flag, optional ε-thinning

Table 2.1 (continued)

Aspect	Review I	Review II
Front reuse	Pickle file on disk	Front store with graph hash, expiry and weather-drift refresh
Tier 2	Weights and argmax	Adds validation, battery feasibility, weather HOLD, JSON/CSV record
Evaluation	Planned	A* ×3, weighted-sum A*, MOPSO; HV, IGD, spread, payload alignment, 225-tag sweep, ablation
Output	Planned `.waypoints`	QGC WPL 110 and `.plan` export, Gazebo world builder, SITL mission runner
Automated tests	11	About 265 (planner and simulation suites)

2.2 Query 2: Phase I Running Until December Is Not Justified

Query. Given the complexity seen at Review I, the time allotted to Phase I, which runs until December, could not be justified.

Answer. With the changes above, the core of the system is now in place. Two pieces of work remain before Phase I can close: benchmarking and evaluation against other algorithms, and bringing the software simulation close to real flying conditions. Neither is a one-off task. Both are loops, and each pass through a loop produces changes that send us back to the start.

2.2.1 Benchmarking and evaluation is a repeated cycle

Figure 2.1 shows the cycle. A scenario set is fixed, every method is run on it, the fronts are scored, the Tier 2 picks are checked, the routes are flown in simulation, and the weak spots are analysed. Whatever we change in response (a cost term, a weight, the normalisation, a parameter of the search) changes every number, so the whole cycle runs again. We stop only when the results are stable and the comparison with each baseline holds up.

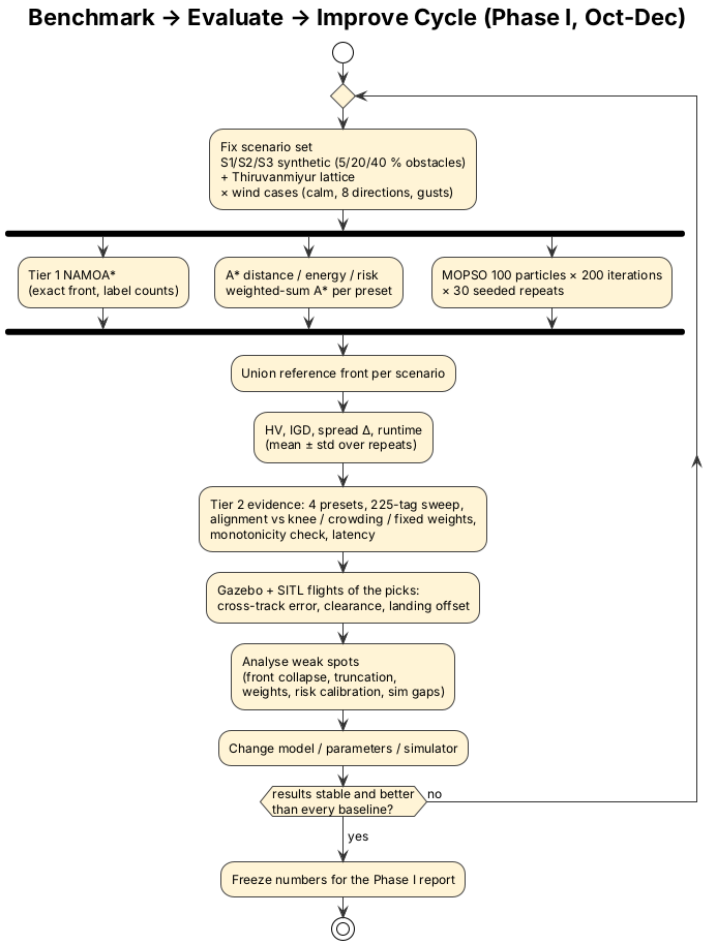


Figure 2.1: The benchmark, evaluate and improve cycle planned for October to December

One pass is not small. The scenario set has four graphs (the S1, S2 and S3 synthetic scenarios plus the real Thiruvanmiyur lattice) under nine wind cases (calm and eight compass directions), which is 36 cases. In each case we compute the exact NAMOA* front, three single-objective A* routes, one weighted-sum A* route per payload preset, and the MOPSO baseline [16] at the settings promised in Review I: 100 particles, 200 iterations and 30 seeded repetitions. That is 600,000 particle updates per case, each with a shortest-path repair, or about 21.6 million per pass. On the Tier 2 side, each case runs all 225 tag combinations and four competing selection rules (shortest route, knee point, NSGA-II crowding distance as in Ren et al. [10], and fixed equal weights in the style of Chokkalingam and Ganesan [2]), which is 8,100 selections per rule per pass.

A first pass has already been run on the synthetic scenarios with reduced MOPSO settings, and it shows why more than one pass is needed. It surfaced the following problems, each of which needs a fix and a full re-run:

(a) The obstacle density labels (5%, 20%, 40%) did not reach the graph builder; the

runner scaled the final risk instead, so the scenarios were not the ones described.

(b) The monotonicity check mixed scenarios and reported 660 false violations on the smallest run; per scenario, the count is zero.

(c) MOPSO repetitions were pooled into one archive, so there is no mean and standard deviation for hypervolume, IGD or spread yet.

(d) Under wind, three of the four payload presets converged on the same route on the benchmark graph (Section 6.3). Whether that is the right answer or a side effect of min-max normalisation on a small front is an open question, and the answer may change Tier 2.

(e) The risk weights and battery constants are still placeholders and must be calibrated before any number is final.

The margins against the baselines are also thin in places. On the 200-node scenario, NAMOA* and MOPSO reach hypervolumes of 1.2065 and 1.2061. A gap that small needs 30 seeded repetitions and a proper statistical test before we can claim that our method is better, and if it turns out not to be, the algorithm has to be improved and the cycle run again. We expect several passes before the numbers are ready for the Phase I report.

2.2.2 Bringing the simulation closer to real conditions

The simulation pipeline is built, and part of it has been tested. The mission runner has flown planner missions on a real ArduCopter SITL build [17]: the upload and read-back, the switch to AUTO, the landing and the disarm all worked, with a cross-track root-mean-square error of 0.11–0.13 m and a landing offset of at most 0.02 m. Those runs used ArduPilot's built-in physics, though, not the city model. The work that remains is about fidelity:

- The Gazebo world [18] has to be built from a live OpenStreetMap and DEM bundle and pass its gates (the world loads, the drone rests on roofs and ground correctly, and the real-time factor stays at or above 0.8).
- The Gazebo world has no wind today, while the planner now plans for wind. Until wind is added to the world and we verify that it actually reaches the vehicle, a comparison of wind cases in simulation would be meaningless.
- The battery model uses placeholder constants. They have to be calibrated from simulated flights for each payload weight class so that the feasibility filter in Tier 2 reflects a real F450-class airframe.

- Each payload preset has to be flown under several wind cases, with cross-track error, clearance, landing offset and end-to-end latency logged for every run.

Every mismatch between the planned and the flown route feeds back into the planner's cost model, which brings us back to the benchmark cycle. The two loops are coupled, so they have to run side by side.

2.2.3 Revised Phase I plan

Figure 2.2 shows the revised timeline. The graph upgrade, the wind model and the hardening of both tiers ran from mid-August to Review II. The benchmark cycles and the simulation work run in parallel from October to December, and the Phase I report is written in December. The hardware phase (Raspberry Pi 5 and Pixhawk integration, outdoor flights) keeps its January to March slot. We believe this schedule is justified by the complexity added since Review I and by the iterative nature of the work that remains.

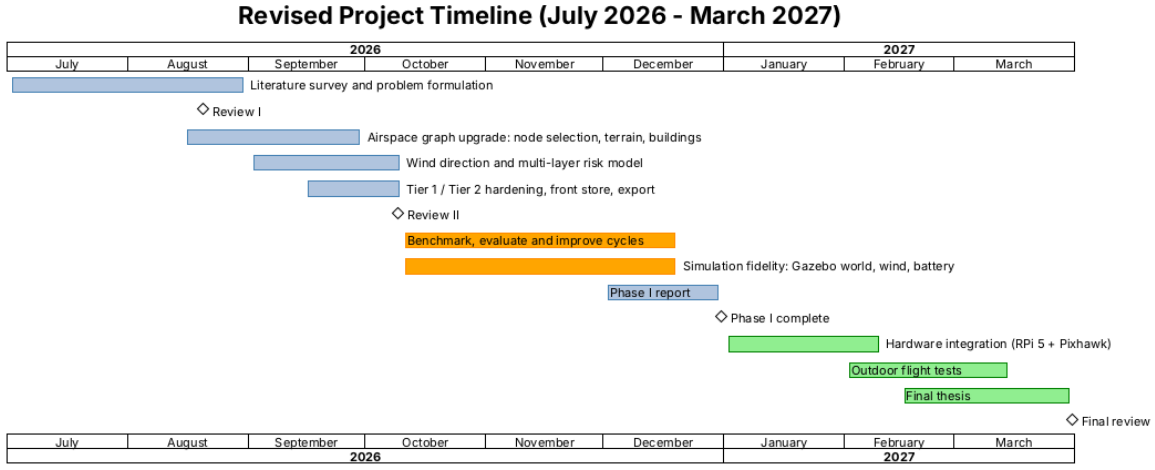


Figure 2.2: Revised project timeline, July 2026 to March 2027

3. Problem Statement

Existing UAV route planners either apply a fixed geometric selection criterion over the Pareto front, independent of payload type (Gap 1), or fold objectives into a single hand-tuned weighted-sum scalar before path search begins, which fixes the objective trade-off regardless of the payload. When the payload context changes, from fragile diagnostic samples that need vibration-minimal routing to time-critical emergency medicine that needs shortest-path routing, an expert must re-tune the planner weights by hand and re-run the search from scratch (Gap 2), because no algebraic mapping

from payload description to routing trade-off exists to make that re-tuning unnecessary (Gap 3). In field conditions, that intervention is not feasible.

Formally, given: (i) an airspace graph $G = (V, E)$ with three-objective edge cost vectors $\mathbf{c}(e) = [d(e),\, \varepsilon(e),\, \rho(e)]$ representing geodesic distance, estimated energy consumption, and flight risk respectively; (ii) a Pareto-optimal front $\mathcal{P}$ of non-dominated routes pre-computed over G; and (iii) a payload metadata tuple $(F, W, U, P_{\text{lvl}})$ encoding fragility class $F \in \{1 \dots 5\}$, payload weight class $W \in \{1 \dots 3\}$, delivery urgency $U \in \{1 \dots 3\}$, and priority level $P_{\text{lvl}} \in \{1 \dots 5\}$, the problem is to select the route $p^* \in \mathcal{P}$ that best satisfies the routing requirements implied by the payload metadata, without expert re-tuning and without re-running the multi-objective search.

The inputs are measurable quantities the operator can assign from domain knowledge (e.g., "this vial is fragile: $F = 5$; delivery is non-urgent: $U = 1$"). The constraints are that selection must be deterministic, reproducible, and finish within one second of payload tag entry. The success criterion is that the selected route's characteristics (distance, energy, risk) match the payload's stated requirements, checked by a Tier 2 selection accuracy metric evaluated across multiple payload presets. Three conditions follow from the gaps. The selected route p^* must depend on $(F, W, U, P_{\text{lvl}})$, not on front geometry alone (Gap 1). The front $\mathcal{P}$ must be reused across missions between scheduled refreshes instead of rebuilt each time (Gap 2). The map from $(F, W, U, P_{\text{lvl}})$ to p^* must be an explicit algebraic formula, not an expert-set rule or a lookup table (Gap 3).

3.1 Weather Dependence of the Problem

The edge costs are functions of the weather. Writing ω for a weather snapshot (wind speed, wind direction, gust, rain, visibility), the cost of an edge is $\mathbf{c}(e; \omega) = [d(e),\, \varepsilon(e; \omega),\, \rho(e; \omega)]$ and the front is $\mathcal{P}(\omega)$. Distance stays fixed, but energy and risk depend on how the wind meets each edge. This does not change the selection problem itself. It does mean that a stored front is only valid while ω stays close to the snapshot it was built for, which is why the front is refreshed on a schedule and on weather drift. If ω breaches the go/no-go limits, no route is selected at all and the mission is held.

3.2 Inputs, Outputs and Success Criteria

Table 3.1 lists what the system takes in, what it returns, and how each part of the problem is judged.

Table 3.1: Inputs, outputs, constraints and success criteria

Item	**Definition**
Region input	Centre point and box size; terrain (DEM), OSM buildings, restricted zones, water and land use for that box
Weather input	Snapshot ω from Open-Meteo (wind at 80 m, gust, rain, visibility, WMO code)
Mission input	Depot, destination and payload tuple $(F, W, U, P_{\text{lvl}})$
Tier 1 output	Stored Pareto front $\mathcal{P}(\omega)$ with graph hash, weather snapshot and expiry
Tier 2 output	Selected route p^* with waypoints (lat, lon, altitude), weights, ranks and point type, or a HOLD advisory
Mission output	QGC WPL 110 `.waypoints` and QGroundControl `.plan` files
Hard constraints	Cruise at least 60 m above ground and 15 m above roofs; at most 120 m above ground; no entry into restricted zones; route energy within 80% of battery capacity
Tier 1 criteria	Hypervolume and IGD at least as good as every baseline on each scenario; no dominated or duplicate routes in the front
Tier 2 criteria	Selection under 1 s; zero monotonicity violations over all 225 tag combinations; non-extreme picks for mixed payloads
Simulation criteria	Cross-track error, clearance margin and landing offset of the flown route in Gazebo and SITL

4. Methodology

The system rests on one design decision: separate Pareto front construction (Tier 1) from route selection (Tier 2), for the reasons in Chapter 2.2.3. The front is built once per delivery region and refreshed periodically, not per mission. A mission that arrives between refreshes runs only Tier 2, which reads the payload tag and returns a route from the current front in a fraction of a second.

The system follows one workflow. Environment data for a delivery region is collected and cached. The airspace graph is built from it, with node and edge selection and a shared cost pipeline. Tier 1 computes the Pareto front and stores it. Tier 2 turns each payload tag into a route, and the route is exported as a mission and flown in simulation. Baseline planners and an evaluation module score every stage, and their findings flow back into the model.

4.1 Architectural Design for Proposed System

Figure 4.1 shows the system architecture. It has five layers. The environment data layer fetches weather, terrain and OpenStreetMap layers and caches them on disk, so that a network failure falls back to the last good copy instead of stopping the planner. Module 1 turns these layers into a directed airspace graph whose edges carry a three-objective cost vector. Tier 1 (Module 2) computes the Pareto front over that graph and keeps it in a front store with the metadata needed to decide when it is stale. Tier 2 (Module 4) reads the operator's payload tags and returns one route from the stored front, or a HOLD advisory when the weather or the battery rules the mission out. The output layer (Modules 6 and 7) writes the mission file and flies it in Gazebo with ArduPilot SITL. Modules 3 and 5 sit beside the main path: they run the baseline planners and compute the quality metrics used in Chapter 5.4.

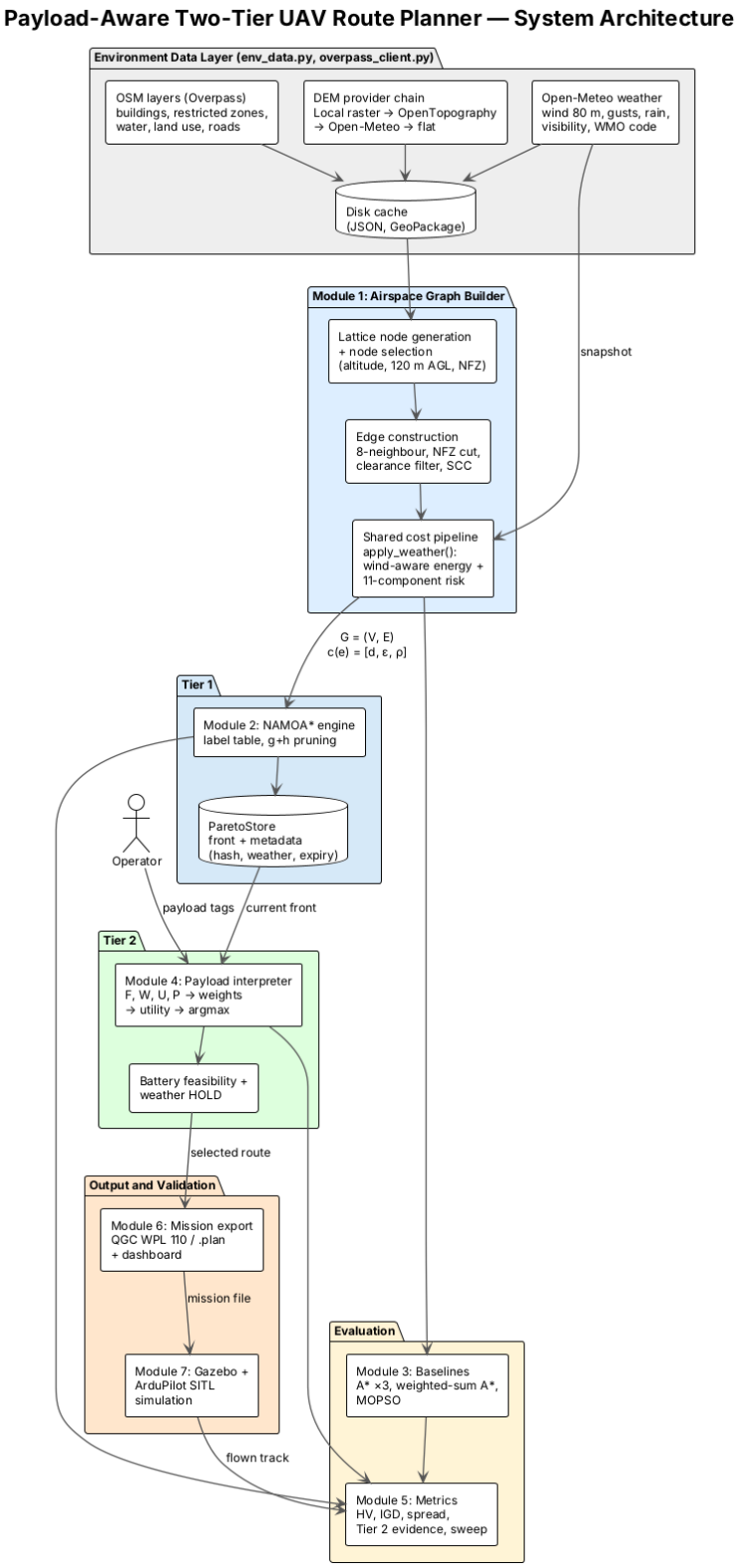


Figure 4.1: System architecture. Tier 1 builds and stores the front per region and weather window; Tier 2 selects a route per payload tag; Modules 6 and 7 export and fly it.

Every edge $e = (u, v)$ of the airspace graph carries the cost vector

$$\mathbf{c}(e; \omega) = \Big[d(e),\ \varepsilon(e; \omega),\ \rho(e; \omega) \Big]$$

where $d(e)$ is the three-dimensional flight distance of the segment in kilometres, $\varepsilon(e; \omega)$ is the wind- and grade-aware energy proxy, and $\rho(e; \omega)$ is the cumulative risk exposure along the segment under weather snapshot ω. Section 4.2.1 defines all three.

The three optimisation objectives are: (1) minimise route distance d; (2) minimise energy consumption ε; and (3) minimise flight risk ρ. A route p dominates route q if p is at least as good as q on all three objectives and strictly better on at least one. The Pareto front $\mathcal{P}$ is the set of all non-dominated routes.

Figure 4.2 follows a single mission through the system, from the moment the operator enters the payload tags to the moment the drone lands.

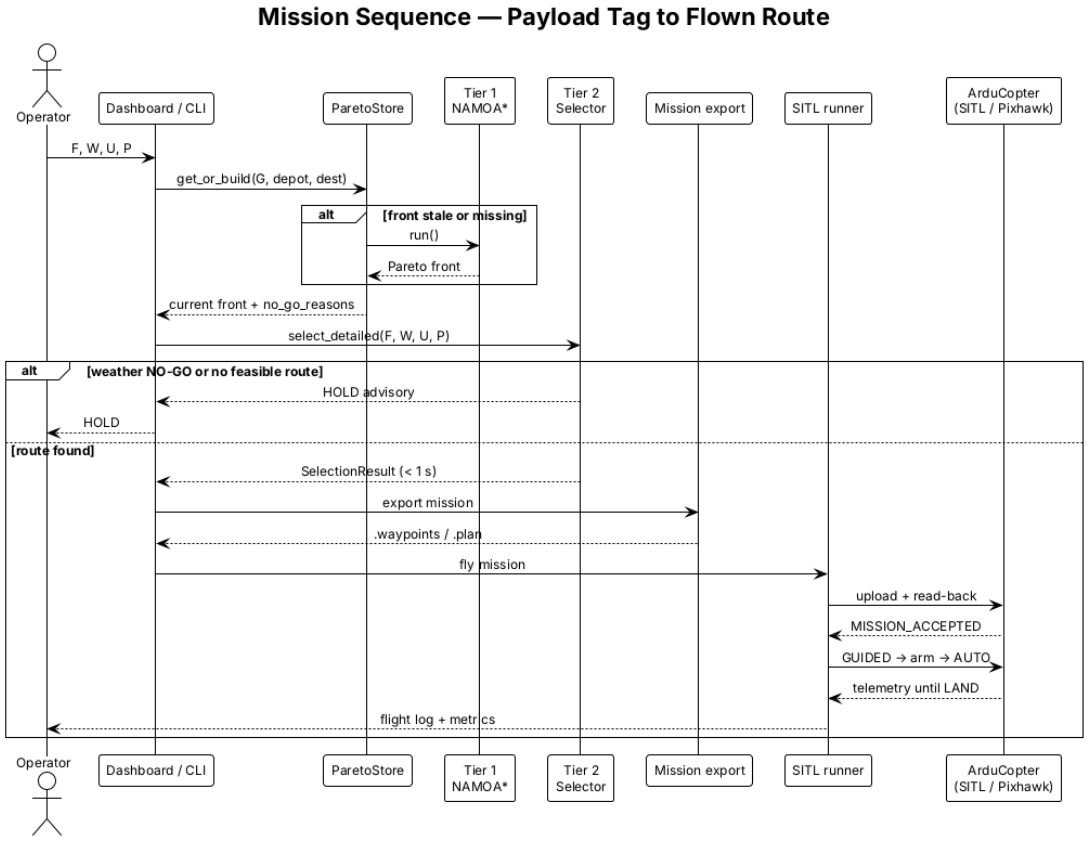


Figure 4.2: Mission sequence from payload tag entry to the flown route

4.2 Modules

The code is organised into seven modules. The numbering follows the source files, so Module 3 holds the baselines and Module 4 holds Tier 2. Each module is described

below in terms of its purpose, inputs, processing, outputs and dependencies.

4.2.1 Module 1: Airspace Graph Builder

Purpose, inputs and outputs

Module 1 builds the delivery airspace graph $G = (V, E)$. Its inputs are the region (centre, box size and lattice spacing), the environment layers described next, and a weather snapshot ω. Its output is a directed NetworkX graph whose nodes carry position and altitude and whose edges carry the static features, the per-component risk breakdown, and the cost vector $[d, \varepsilon, \rho]$. The global weather go/no-go state is stored on the graph itself so that Tier 2 can read it. Two builders share the same cost pipeline: a lattice builder over a real region (Thiruvanmiyur, Chennai) and a seeded synthetic builder used for controlled benchmarks. Figure 4.3 gives the whole pipeline.

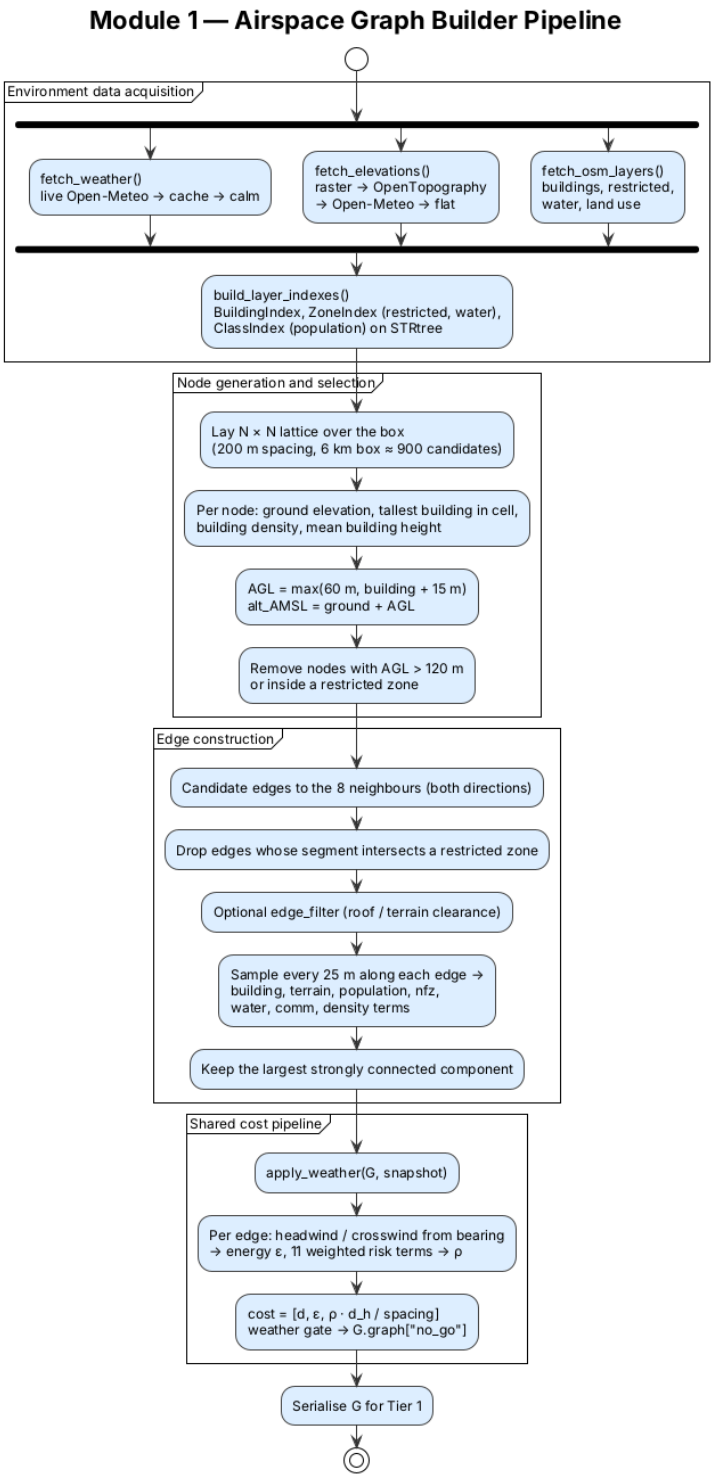


Figure 4.3: Module 1 pipeline: data acquisition, node selection, edge construction and the shared cost pipeline

Environment data acquisition

Weather comes from the Open-Meteo forecast API [11], which needs no key. The module reads the current 10 m wind, gust, precipitation, visibility and WMO weather

code, and the hourly 80 m wind speed and direction. The hourly row is chosen by matching the response's own local time string, so no time-zone conversion is involved. Results are cached; with no network the last cached snapshot is used, and with no cache the planner assumes calm air so that runs stay deterministic.

Terrain comes from an elevation provider chain tried in order: a local GeoTIFF if one is configured, the OpenTopography Copernicus 30 m service (with an API key), the Open-Meteo elevation service (Copernicus 90 m, batches of 100 points), and finally flat ground with a warning. Every value is cached under a key rounded to about one metre.

OpenStreetMap layers are fetched through a multi-endpoint Overpass client with retries, a busy-wait cap and an identifying user agent. Four layers are extracted: building footprints, restricted zones, water bodies (including a sea polygon built from the coastline) and land use with points of interest and major roads. They are cached as GeoPackage files and indexed in local metric coordinates with Shapely STRtree structures: a building index (heights, cell maxima, density, mean height), a zone index for restricted areas and another for water (containment, distance, segment intersection), and a class index for population exposure.

Lattice generation and node selection

The region is covered by an $n \times n$ lattice in a local equirectangular frame centred on the region, with $n = \mathrm{round}(\mathit{box}/\mathit{spacing})$. The default 6 km box at 200 m spacing gives 900 candidate nodes. Each candidate is then put through the selection rule of Figure 4.4. For node k with ground elevation z_k and tallest building b_k in its cell,

$$\mathrm{AGL}_k = \max\Big(H_{\mathrm{cruise}},\ b_k + C\Big), \qquad \mathrm{alt}_k = z_k + \mathrm{AGL}_k \tag{4.1}$$

with $H_{\mathrm{cruise}} = 60\,\mathrm{m}$ and clearance $C = 15\,\mathrm{m}$. A node with $\mathrm{AGL}_k > 120\,\mathrm{m}$ is blocked, because it cannot be flown legally under the Drone Rules, 2021 [12]. A node inside a restricted polygon is removed. Every surviving node stores its position in both frames, its ground elevation, the building height, density and mean height of its cell, its height above ground and its altitude above mean sea level.

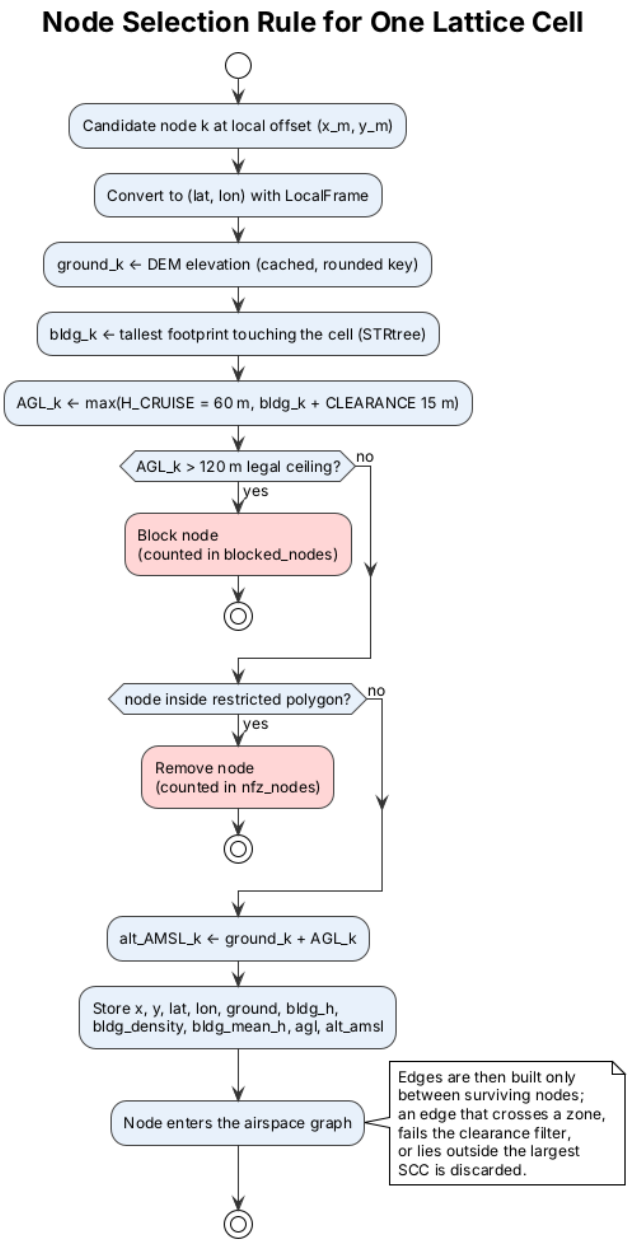


Figure 4.4: Selection rule applied to each candidate lattice node

Edge construction and pruning

Each surviving node is linked to its eight neighbours in both directions, so the graph is directed from the start. Three filters then remove edges. The first drops any edge whose straight segment intersects a restricted polygon, which stops a route from clipping the corner of a zone between two legal nodes. The second is an optional hook, `edge_filter`, that the simulation module uses for hard roof and terrain clearance. The third keeps only the largest strongly connected component, so that every remaining node can reach every other.

For each kept edge the horizontal haversine length d_h, the forward-azimuth

bearing θ, the grade $g = \Delta\text{alt}/d_h$ and the three-dimensional length $d = \sqrt{d_h^2 + \Delta\text{alt}^2}$ are stored. The bearing uses

$$\theta = \operatorname{atan2}\Big(\sin\Delta\lambda\cos\varphi_2,\ \cos\varphi_1\sin\varphi_2 - \sin\varphi_1\cos\varphi_2\cos\Delta\lambda\Big) \tag{4.2}$$

so that the east component is scaled by $\cos\varphi$.

Static risk terms

Static terms do not depend on the weather. They are computed once per build by sampling each edge every 25 m and flying the straight line between the two endpoint altitudes. At each sample k the module knows the flight altitude, the ground height, the tallest roof directly below and the tallest roof within 30 m. Table 4.1 lists all eleven components, both static and weather-dependent, with the weight each one carries in the edge score

$$\rho_{\text{edge}} = \operatorname{clip}\Big(\sum_k w_k\,\rho_k,\ 0,\ 1\Big), \qquad \rho(e;\omega) = \rho_{\text{edge}} \cdot \frac{d_h}{s} \tag{4.3}$$

where s is the lattice spacing. Multiplying by d_h/s turns the per-edge score into a cumulative exposure, so a diagonal hop counts $\sqrt{2}$ times an axis hop and the risk of a route grows with the distance flown through risky air.

Table 4.1: Risk components (every $\rho_k \in [0,1]$) and their weights

Component	**Model**	$\boldsymbol{w_k}$
Obstacle	Seeded obstacle discs, $0.9(1 - r/R)$ inside a disc (synthetic graphs only)	1.00
Building	Mean over samples of [roof within 30 m] $\cdot\,(1 - \operatorname{clip}(m/60))$, m = margin above that roof	0.60
Terrain	$\max(1 - \operatorname{clip}(m_{\min}/15),\ \operatorname{clip}(\lvert g\rvert/0.30))$: lowest clearance margin or sudden altitude change	0.30
Wind	$\operatorname{clip}(CW/40)$, crosswind against a 40 km/h saturation speed	0.30
Gust	$\operatorname{clip}\big(0.5\max(0, gust - v)/40\big)$	0.30
Weather	$0.5\operatorname{clip}(rain/7.5) + 0.5\operatorname{clip}(1 - vis/10\,\text{km})$, uniform over the graph	0.20
Population	Mean SORA-style class along the edge [13]: assembly 1.0, commercial 0.7, major road 0.6, residential 0.5, industrial 0.3	0.50
No-fly buffer	$\max_k \exp(-\delta_k/150\,\text{m})$ to the nearest restricted zone, zero beyond 600 m	0.80
Water	Fraction of samples over water (risk of payload loss)	0.30
Comm/GPS	$\operatorname{clip}(density/0.30) \cdot \operatorname{clip}(\bar{h}/30\,\text{m})$, urban canyon	0.20

Table 4.1 (continued)

Component	**Model**	w_k
Turbulence	$\mathrm{clip}(\mathit{density}/0.30) \cdot \mathrm{clip}(v/40)$	0.20

The weights are design values chosen to put restricted zones, buildings and crowds ahead of weather terms. They have not been calibrated against flight data yet, and calibrating them is part of the evaluation cycle.

Wind-aware energy model

For an edge on bearing θ_{edge} and a wind of speed v blowing from θ_{from} (the meteorological convention), the headwind and crosswind are

$$HW = v\cos(\theta_{\text{edge}} - \theta_{\text{from}}), \qquad CW = v\,|\sin(\theta_{\text{edge}} - \theta_{\text{from}})| \tag{4.4}$$

A positive HW is a headwind. The energy proxy of the edge is

$$\varepsilon = d\left(1 + |g| + \beta_h \frac{\max(0, HW)}{V_c} + \beta_c \frac{CW}{V_c}\right) \tag{4.5}$$

with cruise airspeed $V_c = 36\,\mathrm{km/h}$, $\beta_h = 0.40$ and $\beta_c = 0.15$. A tailwind is clamped to zero benefit. This is a deliberate choice: it keeps $\varepsilon \geq d$ on every edge, which the Tier 1 heuristic relies on. The wind speed itself is the 80 m forecast when available, otherwise the 10 m value lifted to cruise height with the power law $v_h = v_{10}(h/10)^{0.25}$ [14]. A gust factor measured at 10 m is applied to the wind at flight height. Figure 4.5 shows the per-edge computation.

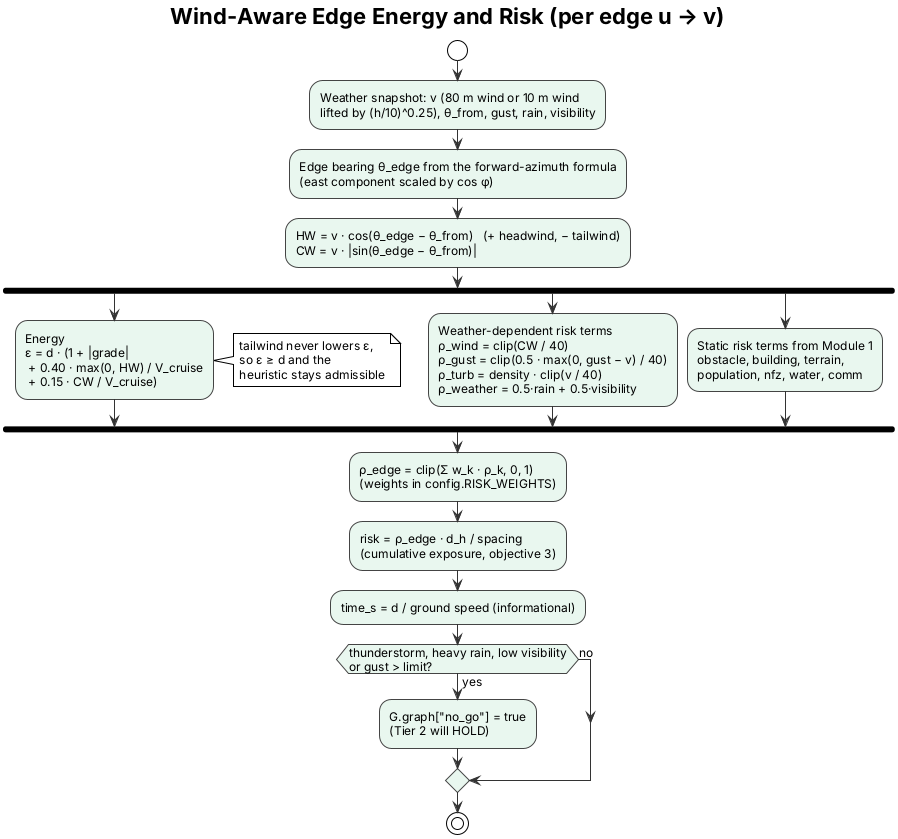


Figure 4.5: Per-edge wind decomposition into energy and risk

Weather-dependent risk and the go/no-go gate

Four of the eleven risk components change with the weather: crosswind, gust excess, turbulence and the uniform rain and visibility term (Table 4.1). Because crosswind depends on the edge heading, the same wind makes some corridors riskier than others, and a change of direction alone can move the low-risk end of the front to a different part of the map. The function `apply_weather(G, ω)` recomputes energy, risk, the component breakdown and an informational flight time on every edge from the stored static features. It is idempotent, so the same graph can be re-costed for a new snapshot without rebuilding any spatial layer.

The weather gate sits on top of this. The mission is a no-go when the WMO code reports a thunderstorm (95–99), when rain reaches 7.5 mm/h, when visibility falls below 1 km, or when gusts exceed 45 km/h, a provisional limit for an F450-class airframe. The reasons are written to the graph and Tier 2 refuses to select a route while any reason is present.

Synthetic scenario builder

Benchmarks need graphs whose difficulty we control. The synthetic builder places nodes on a jittered grid of about 1.3 km spacing with 8-neighbour edges, generates a terrain field from a few seeded Gaussian hills, and drops obstacle discs until a target fraction of the area is covered, with half of them placed along the source-to-target diagonal so that the shortest route has to cross risk. The wind is seeded too (12–28 km/h with a gust factor of 1.4). Every random draw comes from a seed-derived generator, so a scenario is reproduced exactly from its seed. The same `apply_weather` pipeline costs the synthetic edges, which keeps benchmark and real-region results comparable.

4.2.2 Module 2: Tier 1 Pareto Front Engine (NAMOA*)

Purpose, inputs and outputs

Module 2 computes the exact set of non-dominated routes between a source and a target. Its inputs are the costed graph from Module 1, the two end nodes and an expansion cap. Its output is the Pareto front $\mathcal{P}$, a list of cost vectors and node paths, together with diagnostics: whether the search was truncated by the cap, how many labels were expanded and generated, and the runtime.

Labels and dominance

The engine follows NAMOA* [15]. Every partial path is a label $(\mathit{node}, \mathbf{g}, \mathit{parent})$ held in a flat table, and a path is rebuilt at the end by walking parent ids. Nothing is copied when a label is pushed, which matters once the label count runs into thousands. A new label $\mathbf{g}'$ at node v is discarded if any existing label at v weakly dominates it ($\leq$ in every objective within a small tolerance). Equal vectors count as dominated, so the front never contains duplicates. Labels that the new one dominates are marked dead and skipped when popped. No visited set is needed: every edge has positive distance, so a path that revisits a node is dominated by its own earlier label at that node, and cycles die in the dominance check.

Admissible vector heuristic and pruning

The heuristic for node v is $\mathbf{h}(v) = [h, h, 0]$, where h is the great-circle distance from v to the target. It is admissible for all three objectives. For distance, every edge satisfies $d \geq d_h$, and the horizontal lengths of any path to the target sum to at least h. For energy, Equation (4.5) gives $\varepsilon \geq d$, so the same bound holds. For risk, $\rho \geq 0$. A label

is pruned when the current front weakly dominates $\mathbf{g} + \mathbf{h}$, both when it is generated and again when it is popped, because the front may have improved in between. The open list is ordered by $\|\mathbf{g} + \mathbf{h}\|$, which tends to complete good routes early and so tightens the pruning sooner.

Truncation and front thinning

The expansion cap protects the planner from a runaway search on a dense graph. When the cap is hit, the engine checks whether any live label could still improve the front after adding its heuristic. Only then is the result flagged as truncated, and a warning is printed. For large fronts an optional ε-dominance thinning step can cap the front at N routes. Objectives are min-max normalised and cut into ε-boxes, one route survives per non-dominated box, ε is found by bisection, and the three single-objective extremes are always kept because Tier 2 normalises against them. Thinning is off by default so that benchmarks use the exact front.

Algorithm 1 NAMOA* with a label table (Tier 1)

Require: Directed graph $G = (V, E)$, source s, target t, costs $\mathbf{c}(u, v) = [d, \varepsilon, \rho]$, cap M

Ensure: Pareto front $\mathcal{P}$, flag *truncated*

$L \leftarrow [(s, [0, 0, 0], -1)]$; $at[s] \leftarrow \{0\}$; $F \leftarrow \emptyset$; $Open \leftarrow \{(\|\mathbf{h}(s)\|, 0)\}$
while $Open \neq \emptyset$ **do**
  if $expanded \geq M$ **then**
    $truncated \leftarrow \exists$ live $\ell \in Open$ with $F \not\preceq \mathbf{g}_\ell + \mathbf{h}(u_\ell)$; **break**
  end if
  $\ell = (u, \mathbf{g}) \leftarrow$ PopMin($Open$)
  if ℓ is dead **or** $F \preceq \mathbf{g} + \mathbf{h}(u)$ **then continue**
  end if
  $expanded \leftarrow expanded + 1$
  for each successor v of u **do**
    $\mathbf{g}' \leftarrow \mathbf{g} + \mathbf{c}(u, v)$
    if $\exists\, o \in at[v] : \mathbf{g}_o \preceq \mathbf{g}'$ **then continue** ▷ node-level dominance
    end if
    if $F \preceq \mathbf{g}' + \mathbf{h}(v)$ **then continue** ▷ front pruning on $g + h$
    end if
    mark dead every $o \in at[v]$ with $\mathbf{g}' \preceq \mathbf{g}_o$
    append label $(v, \mathbf{g}', \ell)$ to L and to $at[v]$
    if $v = t$ **then**
      $F \leftarrow \{f \in F : \mathbf{g}' \not\preceq f\} \cup \{\mathbf{g}'\}$
    else
      push $(\|\mathbf{g}' + \mathbf{h}(v)\|, \text{label})$ onto $Open$
    end if
  end for
end while
rebuild each route in F by following parent ids; **return** $\mathcal{P}$, *truncated*

Here $\preceq$ is weak dominance. Figure 4.6 gives the same procedure as an activity diagram.

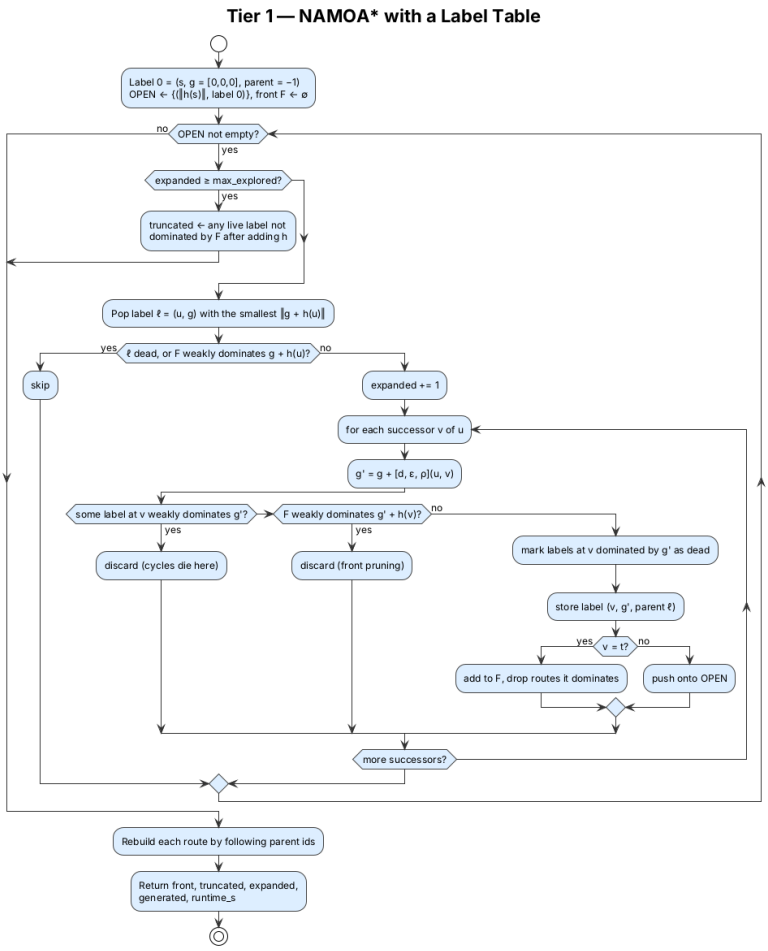


Figure 4.6: Tier 1 activity flow: NAMOA* with a label table

Pareto front store and refresh

The front is built per region and reused, so the system needs a rule for when reuse is safe. The front store saves one JSON record per (region, source, target). Each record holds the front, a SHA-256 hash over the node ids and static edge attributes (never the weather-dependent costs, so a change of weather alone does not alter the hash), the weather snapshot it was built for, the creation time, a validity limit three hours later, the truncation flag and the search diagnostics. An optional identity field ties the front to a simulation bundle.

On each request the store lists the reasons the record cannot be reused (Figure 4.7): no record, a forced refresh, a different static graph, a different identity, expiry, or weather drift. Drift means the wind speed has moved by more than 10 km/h, the direction has turned by more than 45° while the wind is above 10 km/h, the gust has moved by more than 15 km/h, or the go/no-go state has flipped. If there is no reason, Tier 2 runs against the stored front and no search happens. If there is one, only `apply_weather` and Tier 1 run again; the spatial layers stay cached. A helper

also builds one front per delivery point for a depot, snapping each point's latitude and longitude to the nearest lattice node.

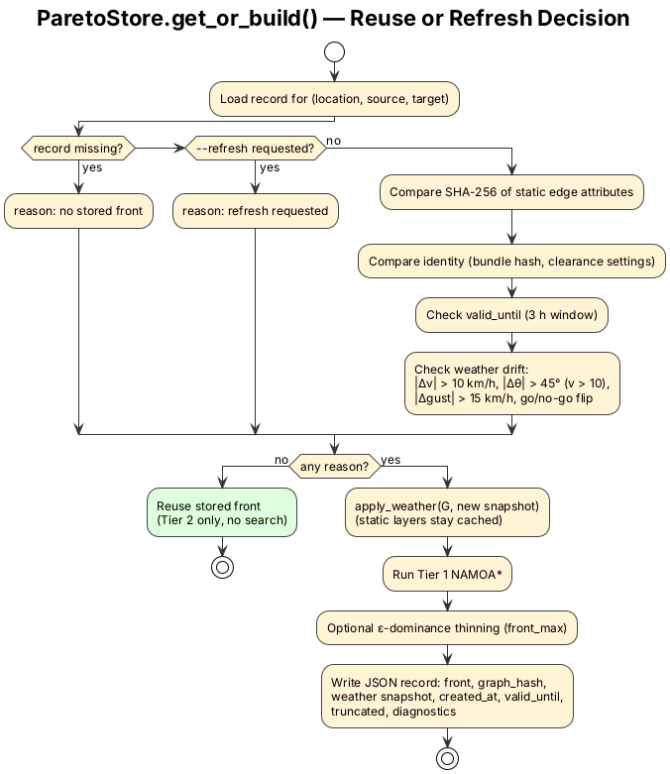


Figure 4.7: Front store decision: reuse the stored front or rebuild it

4.2.3 Module 3: Baseline Planners

Purpose

Module 3 holds the planners that Tier 1 and Tier 2 are compared against. They read the same costed graph, so any difference in the results comes from the search method and not from the data.

Single-objective A*

Three A* searches minimise distance, energy and risk separately. The distance search uses the haversine distance to the target as its heuristic, which is admissible for the same reason as in Tier 1. The energy and risk searches use a zero heuristic, which makes them Dijkstra searches behind the same interface. Each returns one route, the extreme point of the front for its objective. Their purpose in the comparison is to show what a planner that ignores the trade-off would fly.

Weighted-sum A*

The weighted-sum baseline scalarises the cost vector with the Tier 2 weights of a payload preset, $\sum_j w_j c_j(e)/s_j$, where each scale s_j is the spread of objective j across the three single-objective routes, and runs A* on that scalar. This is the classic approach that the problem statement criticises. It can only reach supported points of the front (points on its convex hull), so comparing it with Tier 2 shows whether scoring the full front reaches routes that a single scalarised search cannot.

MOPSO

The population-based baseline adapts multi-objective particle swarm optimisation [16] to graph routes. A particle is a valid source-to-target path. The swarm is seeded with the three single-objective routes, an equal-weight route and biased random walks. In each iteration a particle samples a random preference vector, follows its personal best or a leader drawn from the archive for part of the way, and is repaired to the target with a weighted shortest suffix. Only valid non-dominated paths enter an archive of bounded size. The settings for the reported experiments are 100 particles, 200 iterations, an archive of 100 and 30 seeded repetitions; a smaller smoke setting is used for quick checks.

4.2.4 Module 4: Tier 2 Payload Metadata Interpreter

Purpose, inputs and outputs

Module 4 turns the payload description into a route. Its inputs are the stored front, the weather go/no-go reasons from the graph, and the four tags $(F, W, U, P_{\text{lvl}})$. Its output is a selection record: the tags, the derived weights, the selected route index, cost vector and waypoints with latitude, longitude and altitude, the normalised scores, the selected route's rank on each objective, whether it is an extreme or non-extreme point, the latency, the number of battery-feasible routes and, when no route can be flown, a HOLD advisory. The record is written to JSON and CSV.

Tag validation

Each tag must be an integer in its range: $F \in \{1..5\}$, $W \in \{1..3\}$, $U \in \{1..3\}$, $P_{\text{lvl}} \in \{1..5\}$. Booleans and floats are rejected. A bad tag raises an error before any weight is derived, so a typing mistake at the ground station cannot quietly change the route.

The algebraic bridge

The Tier 2 algebraic bridge maps each of the four payload metadata tags to exactly one Tier 1 routing objective, as shown in Table 4.2.

Table 4.2: Semantic mapping: payload metadata tags to Tier 1 routing objectives

Payload Tag	**Range**	**Maps to Tier 1 Objective**	**Reasoning**
Fragility class F	1–5	w_{risk} (flight risk weight)	Fragile items need low-vibration, low-risk corridors
Payload weight class W	1–3	w_{energy} (energy weight)	Heavier payload increases hover/lift power draw ⇒ higher energy cost
Delivery urgency U	1–3	w_{dist} (distance weight)	Time-critical deliveries need the shortest path
Priority level P_{lvl}	1–5	Tie-breaker amplifier	Scales the dominant weight when scores are close

From the payload tags, three routing objective weights are derived algorithmically with no expert input:

$$r_{\text{raw}} = F \times 2, \quad d_{\text{raw}} = U \times 3, \quad e_{\text{raw}} = W \times 2 \tag{4.6}$$

$$w_{\text{risk}} = \frac{r_{\text{raw}}}{r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}}}, \quad w_{\text{dist}} = \frac{d_{\text{raw}}}{r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}}}, \quad w_{\text{energy}} = \frac{e_{\text{raw}}}{r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}}} \tag{4.7}$$

The multipliers (2 for fragility and payload weight, 3 for urgency) encode the assumption that urgency is more penalty-sensitive than fragility or payload weight in the UAV medical delivery context. Payload weight is tied to the energy objective because a heavier payload directly increases hover and lift power draw; no such direct physical link holds for risk or distance.

For each Pareto point p_i with objective values $(d_i, \varepsilon_i, \rho_i)$, min-max normalised values $\tilde{d}_i$, $\tilde{\varepsilon}_i$, $\tilde{\rho}_i$ are computed over the full Pareto front $\mathcal{P}$. The utility score is:

$$\text{score}(p_i) = w_{\text{dist}} \cdot (1 - \tilde{d}_i) + w_{\text{energy}} \cdot (1 - \tilde{\varepsilon}_i) + w_{\text{risk}} \cdot (1 - \tilde{\rho}_i) \tag{4.8}$$

The selected mission route is $p^* = \arg\max_{p_i \in \mathcal{P}} \text{score}(p_i)$. For high-priority missions ($P_{\text{lvl}} \geq 4$), the dominant weight is amplified by $0.1 \times w_{\max}$ to sharpen the selection.

Table 4.3 gives the weights that the bridge produces for the four reference presets. Only emergency medicine has $P_{\text{lvl}} \geq 4$, so it is the only preset whose dominant weight is amplified.

Table 4.3: Reference payload presets and their derived weights

Preset	F	W	U	P_{lvl}	w_{risk}	w_{dist}	w_{energy}
Fragile medical sample	5	1	1	3	0.667	0.200	0.133
Emergency medicine	1	1	3	5	0.144	0.712	0.144
Disaster relief	1	3	1	2	0.182	0.273	0.545
Mixed cargo	3	2	2	3	0.375	0.375	0.250

Algorithm 2 Payload-Driven Pareto Route Selection (Tier 2)

Require: Pareto front $\mathcal{P} = \{(d_i, \varepsilon_i, \rho_i)\}$, payload tags F, W, U, P_{lvl}
Ensure: Selected mission route p^*

$r_{\text{raw}} \leftarrow F \times 2$; $\quad d_{\text{raw}} \leftarrow U \times 3$; $\quad e_{\text{raw}} \leftarrow W \times 2$
$total \leftarrow r_{\text{raw}} + d_{\text{raw}} + e_{\text{raw}}$
$w_{\text{risk}} \leftarrow r_{\text{raw}}/total$; $\quad w_{\text{dist}} \leftarrow d_{\text{raw}}/total$; $\quad w_{\text{energy}} \leftarrow e_{\text{raw}}/total$
if $P_{\text{lvl}} \geq 4$ **then** ▷ high-priority amplification
    $w_{\max} \leftarrow \max(w_{\text{risk}}, w_{\text{dist}}, w_{\text{energy}})$
    add $0.1 \times w_{\max}$ to whichever of $w_{\text{risk}}, w_{\text{dist}}, w_{\text{energy}}$ equals $w_{\max}$
    renormalise $w_{\text{risk}}, w_{\text{dist}}, w_{\text{energy}}$ so they sum to 1
end if
compute $d_{\min}, d_{\max}, \varepsilon_{\min}, \varepsilon_{\max}, \rho_{\min}, \rho_{\max}$ over $\mathcal{P}$
for each $p_i = (d_i, \varepsilon_i, \rho_i) \in \mathcal{P}$ **do**
    $\tilde{d}_i \leftarrow (d_i - d_{\min})/(d_{\max} - d_{\min})$ ▷ 0 if $d_{\max} = d_{\min}$
    $\tilde{\varepsilon}_i \leftarrow (\varepsilon_i - \varepsilon_{\min})/(\varepsilon_{\max} - \varepsilon_{\min})$
    $\tilde{\rho}_i \leftarrow (\rho_i - \rho_{\min})/(\rho_{\max} - \rho_{\min})$
    $\text{score}(p_i) \leftarrow w_{\text{dist}}(1 - \tilde{d}_i) + w_{\text{energy}}(1 - \tilde{\varepsilon}_i) + w_{\text{risk}}(1 - \tilde{\rho}_i)$
end for
$p^* \leftarrow \arg\max_{p_i \in \mathcal{P}} \text{score}(p_i)$
return p^*

Battery feasibility

The payload weight class also decides whether a route can be flown at all. Each class maps to a nominal payload mass m_W (0.25, 0.5 and 1.0 kg for $W = 1, 2, 3$), and a

route's energy proxy is converted to watt-hours with

$$E_{\mathrm{Wh}} = k_{\mathrm{cal}}\,\varepsilon \left(\frac{m_{\mathrm{uav}} + m_W}{m_{\mathrm{uav}}} \right)^{1.5} \tag{4.9}$$

where the exponent follows the scaling of rotor induced power with weight. With the current values ($k_{\mathrm{cal}} = 6\,\mathrm{Wh}$ per km of energy proxy, $m_{\mathrm{uav}} = 1.5\,\mathrm{kg}$, a 74 Wh battery and a 20% reserve), a route is feasible when it needs at most 59.2 Wh. Infeasible routes are removed before the argmax. These constants are marked as placeholders in the code and will be replaced by values calibrated in simulation.

Weather hold and decision record

If the weather gate reports any reason, or if no route on the front is battery-feasible, Tier 2 returns a HOLD advisory that says why, instead of a route. Otherwise it returns the feasible route with the highest score, breaking ties toward the lower index so the result is deterministic. Figure 4.8 shows the full flow.

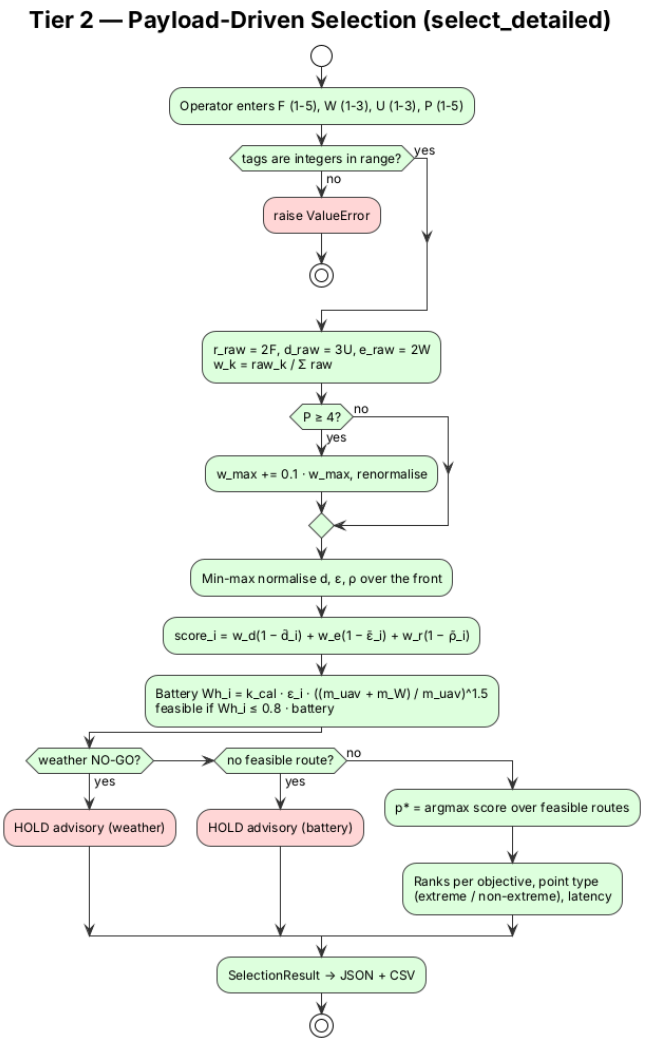


Figure 4.8: Tier 2 activity flow with validation, feasibility and weather hold

4.2.5 Module 5: Evaluation and Benchmarking

Purpose

Module 5 measures how good a front is and how sensible the Tier 2 picks are. Every measure is computed per scenario, and the results are written to CSV and JSON so that each pass of the evaluation cycle can be compared with the last.

Reference front and normalisation

There is no known true front for our graphs, so the reference front $\mathcal{R}$ for a scenario is the non-dominated union of every route found by every method and every repetition. All metrics are computed in a space normalised by the ideal and nadir points of $\mathcal{R}$.

Front quality metrics

Hypervolume (HV) [19] is the volume dominated by a front and bounded by a reference point, here the normalised nadir padded by 10% ($[1.1, 1.1, 1.1]$). It is computed exactly in three dimensions by recursive slicing. Larger is better. Inverted generational distance measures how well a front A covers the reference front:

$$\mathrm{IGD}(A, \mathcal{R}) = \frac{1}{|\mathcal{R}|} \sum_{r \in \mathcal{R}} \min_{a \in A} \|r - a\|_2 \tag{4.10}$$

Smaller is better, and zero means A contains every reference point. The spread Δ captures how evenly the points are distributed and how far they reach toward the extremes of $\mathcal{R}$. Runtime and front size are recorded for every method.

Tier 2 evidence

We do not report a Tier 2 "accuracy", because there is no ground-truth label for the correct route. Instead, for each preset the module records the selected route's rank on each objective, whether it is an extreme or a non-extreme point, the latency, and a rank-based payload-alignment score

$$A = \sum_{k \in \{d, \varepsilon, \rho\}} w_k \left(1 - \frac{\mathrm{rank}_k - 1}{K - 1}\right) \tag{4.11}$$

where K is the front size. Because A uses ranks, it does not depend on the min-max utility that Tier 2 itself maximises. The same score is computed for four competing selection rules on the same front: the shortest route, the knee point (the point farthest from the line joining the distance and risk extremes), the largest NSGA-II crowding distance [9, 10], and a fixed equal-weight utility in the style of [2].

Tag sweep and monotonicity

A sweep runs Tier 2 for all $5 \times 3 \times 3 \times 5 = 225$ tag combinations. It counts distinct selected routes and non-extreme picks, and it checks monotonicity: raising F with the other tags fixed must never raise the selected route's risk, raising U must never raise its distance, and raising W must never raise its energy. The target is zero violations.

Ablation and component shares

An ablation runner rebuilds the real-region front with all layers, without wind, without terrain and without buildings, and draws the edge-risk map and front for each. A component-share table reports the mean share of each risk component across the front

routes and across the selected routes, which shows which layers actually drive the choice.

4.2.6 Module 6: Mission Export and Ground-Station Dashboard

Mission file generation

Module 6 turns a selection record into a mission that ArduPilot ground stations can load. Consecutive lattice waypoints that keep the same bearing and altitude slope (within a small tolerance) are merged, so a straight run of lattice hops becomes one leg. The file follows the QGC WPL 110 text format. The first item is home, in the global frame. Every route waypoint uses frame 3 (`GLOBAL_RELATIVE_ALT`), whose altitude is measured from home, so the exporter writes

$$alt_{\mathrm{rel}} = alt_{\mathrm{AMSL}} - H_{\mathrm{home}} \tag{4.12}$$

with H_{home} the ground elevation under the first waypoint. The height above local ground is never written into frame 3, because that would be wrong wherever the terrain varies. An optional return-to-launch item can close the mission.

Operator dashboard

A small Tkinter dashboard lets an operator load a stored graph and front, type the four tags, see the selected route drawn on the map, and export the mission file. It covers the ground-station objective in a minimal form. Two defects are open: the dashboard does not yet pass the weather reasons to Tier 2, and it can load a graph and a front from different runs. Both are on the list for the next pass.

4.2.7 Module 7: Simulation and SITL Validation

Purpose

A route that scores well on the graph may still be hard to fly, so Module 7 checks the flight itself. It flies the Tier 2 route through a three-dimensional model of the delivery area in Gazebo Harmonic [18], with ArduCopter running in software-in-the-loop [17], and compares the flown track with the plan. Figure 4.9 shows the pipeline. The simulation code imports the planner modules directly; nothing is copied.

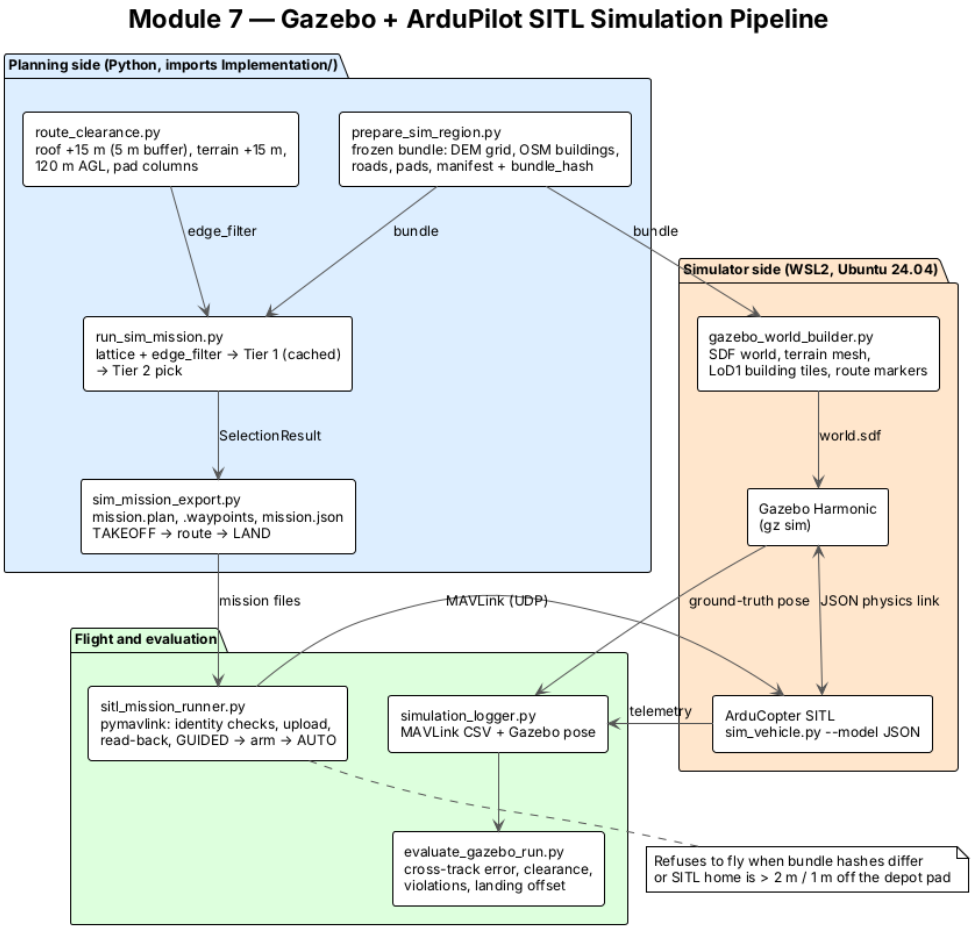


Figure 4.9: Gazebo and ArduPilot SITL simulation pipeline

Frozen region bundle

A region is first frozen into a bundle: a terrain grid, OSM buildings with their heights and the source of each height, roads, the depot and destination pads, and a manifest. The default simulation region is a 1.5 km box with a 100 m lattice. A hash of the bundle is written into the graph, the front identity, the mission file and the world manifest, and the runner refuses to fly if they do not match. This stops a mission planned on one version of the map from being flown in another.

Hard clearance filter

Before NAMOA* runs, a clearance checker is passed to the lattice builder as its `edge_filter`. It removes every edge whose buffered segment (5 m) passes less than 15 m above a roof, whose terrain clearance sampled every 10 m drops below 15 m, or which exceeds 120 m above ground. The take-off and landing columns over the pads, and the connector legs from the pads to the lattice, are checked the same way. Waypoints are never raised after selection. If a check fails, the planner reports that no

feasible mission exists and writes nothing.

World building and frame contract

The world builder writes a Gazebo SDF world with a terrain mesh, LoD1 building tiles (footprints extruded to their heights), roads, pads and route markers. The world origin is the depot pad on the ground, with east, north and up axes. SITL is started with its home at the same point, using the same terrain grid as the planner, so the mission altitude $alt_{\mathrm{AMSL}} - H_{\mathrm{home}}$ means the same height in the planner, the world and the autopilot. The runner checks this before arming and refuses to fly if home is more than 2 m horizontally or 1 m vertically from the depot pad.

Mission flight and logging

The simulation mission is take-off, the route, and a landing on the destination pad, with no unvalidated return leg. The runner, built on pymavlink, performs identity checks, uploads the mission, reads it back and compares it with what was sent, switches to GUIDED, arms, switches to AUTO, and logs telemetry until the vehicle lands and disarms. A logger records the MAVLink stream and the Gazebo ground-truth pose side by side.

Flight evaluation

The evaluator computes the cross-track error of the flown track against the planned polyline, the minimum clearance above roofs and terrain, any violation of the clearance or ceiling rules, and the landing offset from the destination pad. It draws the map and altitude plots and appends a row to a summary table, so runs for different presets and wind cases can be compared.

5. Implementation

The planner is written in Python as flat scripts under `Implementation/`, one file per module, plus a configuration file, an environment-data file and entry-point scripts. The simulation lives under `Simulation/` and imports the planner modules through the Python path. This chapter describes how each module is implemented, the data structures it uses and how it is tested.

5.1 Development Environment and Tools

Table 5.1: Languages, libraries and tools

Tool	**Use in the project**
Python 3	All planner, evaluation and simulation-side code
NetworkX [20]	Directed airspace graph, A* baselines, strongly connected components
NumPy	Vectorised node environment, dominance checks, metrics
Shapely 2 (STRtree)	Spatial indexes for buildings, zones, water and population classes
GeoPandas, pyogrio, OSMnx [21]	OSM layer parsing and GeoPackage caching
requests	Open-Meteo, OpenTopography and Overpass HTTP calls
Matplotlib	Graph, front, route and benchmark figures
Tkinter	Operator dashboard
pymavlink, MAVLink [22]	Mission upload, read-back and flight control in SITL
Gazebo Harmonic, ArduPilot SITL	Three-dimensional city simulation and autopilot
unittest	Planner and simulation test suites (offline, keyless)
PlantUML, LaTeX	Diagrams and this report

All tunable constants (region, flight heights, wind coefficients, risk weights, battery and mass values, drift thresholds, Overpass endpoints) live in one file, `config.py`, so an experiment changes a value in one place. API keys are read from a `.env` file by the entry-point scripts only, so the test suites never need a key or a network connection.

5.2 Code Organisation

Table 5.2: Source files and the modules they implement

File	Module	Role
`config.py`	all	Every tunable constant
`env_data.py`	1	Weather, DEM providers, OSM layers, spatial indexes, `.env` loader
`overpass_client.py`	1	Multi-endpoint Overpass fetch with retry
`module1_graph_builder.py`	1	Synthetic and lattice builders, cost pipeline
`module2_tier1_moastar.py`	2	NAMOA* engine, dominance helpers, thinning
`pareto_store.py`	2	Front store, graph hash, drift, region runs
`module3_baselines.py`	3	A* variants, weighted-sum A*, MOPSO
`module4_tier2_payload.py`	4	Weight bridge, selector, feasibility, export
`module5_evaluation.py`	5	HV, IGD, spread, Tier 2 evidence, sweep
`run_benchmarks.py`, `run_ablation.py`	5	Benchmark and ablation runners
`mission_export.py`, `dashboard.py`	6	QGC WPL 110 export, operator UI
`run_demo_syn.py`, `run_demo_osm.py`	1–4	End-to-end demos with figures
`Simulation/*.py`	7	Bundle, clearance, world, mission, runner, logger, evaluator

Figure 5.1 shows the main classes and how they relate.

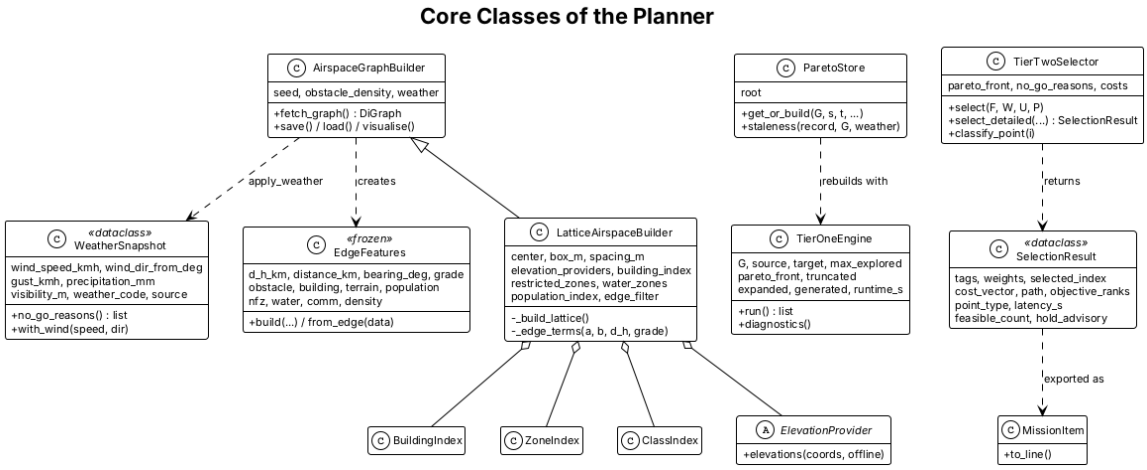


Figure 5.1: Core classes of the planner

5.3 Module 1: Airspace Graph Builder

5.3.1 Weather and terrain data

`WeatherSnapshot` is a dataclass holding wind speed, direction, gust, rain, visibility, the WMO code, the observation time and the source of the data (live, power-law, cache, override, synthetic or calm). Its constructor normalises the direction to $[0, 360)$ and forces the gust to be at least the mean wind. `no_go_reasons()` implements the weather gate, and `with_wind()` produces the override used for controlled wind experiments. `fetch_weather()` never raises: it tries the live API, then the cache, then calm air.

Elevation providers share one interface, the `elevations()` method of `ElevationProvider`, which looks up the disk cache first and only sends uncached points to the service. `fetch_elevations()` walks the provider list and reports which provider supplied the values, so the console always says where the terrain came from.

5.3.2 OSM layers and spatial indexes

`fetch_osm_layers()` queries the Overpass endpoints in turn (with two attempts each, a pause on HTTP 406, 429 and 5xx, and a cap on `Retry-After` waits) and caches the result as GeoPackage. A failed fetch returns empty layers and the run still completes, so the console prints the layer counts and they must be checked before a result is called real. `build_layer_indexes()` projects every geometry into the local metric frame and builds the indexes. `BuildingIndex` answers four vectorised queries over arrays of points: the tallest building in a square cell, the building density and mean height of a cell, the tallest building directly under a point, and the tallest building within a radius. `ZoneIndex` answers containment, clipped distance and segment-intersection queries, and `ClassIndex` returns the highest population class covering a point.

5.3.3 Lattice builder

`LatticeAirspaceBuilder._build_lattice()` works on whole arrays. It builds the node coordinates with a NumPy mesh grid, converts them to latitude and longitude in one call, fetches all ground elevations in one batched provider call and computes the building statistics for all cells at once. The blocked mask $(\text{AGL} > 120) \vee \text{inNFZ}$ is a single array expression. Candidate edges are collected first, and the NFZ test on all of them runs as one vectorised segment-intersection query. The `edge_filter` hook is then applied, the static terms are computed per edge by `_edge_terms()`, and the graph is reduced to its largest strongly connected component. Build statistics

(provider, layer counts, blocked nodes, removed edges) are stored on the graph for the report and the console.

Each edge's static features are held in a frozen dataclass, `EdgeFeatures`, which clips every risk term to $[0, 1]$ when it is created. Weather-dependent values are never stored in it. The cost pipeline is a pair of pure functions:

```
def wind_components(bearing_deg, speed_kmh, dir_from_deg):
    rel = math.radians(bearing_deg - dir_from_deg)
    return speed_kmh * math.cos(rel), speed_kmh * abs(math.sin(rel))

def energy_cost(f: EdgeFeatures, w: WeatherSnapshot) -> float:
    hw, cw = wind_components(f.bearing_deg, w.wind_speed_kmh,
                             w.wind_dir_from_deg)
    factor = (1.0 + abs(f.grade)
              + cfg.BETA_HEAD * max(0.0, hw) / cfg.V_CRUISE_KMH
              + cfg.BETA_CROSS * cw / cfg.V_CRUISE_KMH)
    return f.distance_km * factor
```

Listing 5.1: Wind decomposition and energy (module1_graph_builder.py)

The function `apply_weather()` loops over the edges, rebuilds the static features of each edge from its stored attributes, and writes `energy`, `rho`, `risk`, `risk_components`, `cost` and `time_s`. It then writes the snapshot and the gate result to `G.graph`.

5.3.4 Main parameters

Table 5.3: Main Module 1 parameters (`config.py`)

Parameter	Value	Meaning
`CENTER_LAT`, `CENTER_LON`	12.9830, 80.2594	Thiruvanmiyur, Chennai
`BOX_M`, `LATTICE_SPACING_M`	6000 m, 200 m	Lattice extent and spacing
`H_CRUISE_AGL`, `CLEARANCE`	60 m, 15 m	Cruise height, margin above roofs
`H_MAX_AGL`	120 m	Legal ceiling
`V_CRUISE_KMH`	36 km/h	Airspeed used in the wind terms
`BETA_HEAD`, `BETA_CROSS`	0.40, 0.15	Headwind and crosswind energy coefficients
`V_MAX_SAFE_KMH`, `K_GUST`	40 km/h, 0.5	Wind risk saturation, gust weight
`ALPHA_SHEAR`	0.25	Power-law shear exponent
`EDGE_SAMPLE_STEP_M`	25 m	Along-edge sampling step
`NFZ_SOFT_LAMBDA_M`	150 m	Decay length of the no-fly buffer
`NO_GO_GUST_KMH`	45 km/h	Gust limit (provisional)

5.3.5 Entry points

`run_demo_osm.py` runs the lattice pipeline end to end. Its options cover the experiments in this report: `-offline` (cache or fixtures only), `-wind-override SPEED DIR` (controlled wind), `-no-terrain`, `-no-buildings` and `-no-risk-layers` (ablation), `-refresh` (force a new front), `-front-max N` (thinning), `-box-km` and `-spacing-m` (graph size), and `-payload F W U P` or `-preset` (Tier 2 input). `run_demo_syn.py` does the same on the synthetic graph.

5.4 Module 2: Tier 1 Engine and Front Store

5.4.1 Label table

`TierOneEngine.run()` keeps the label table as four parallel Python lists (`lab_node`, `lab_g`, `lab_parent`, `lab_dead`) indexed by label id, and a dictionary from node to the ids of its live labels. The open list is a binary heap of (priority, sequence number, label id); the sequence number breaks ties so that runs are deterministic. Heuristic vectors are cached per node. The front is kept twice: as a list of label ids and as a NumPy array, so that the check "does the front weakly dominate this vector" is one

vectorised comparison instead of a Python loop. When the search ends, each front label is turned into a path by following parent ids, and the routes are sorted by cost vector.

5.4.2 Diagnostics

After a run the engine exposes `truncated`, `expanded`, `generated`, `runtime_s` and the front size through `diagnostics()`. The default cap is 200,000 expansions; the benchmark runner raises it per scenario (80,000, 180,000 and 260,000 for S1, S2 and S3).

5.4.3 Front store

`ParetoStore` writes one indented JSON file per (region, source, target) under `pareto_front_storage/fronts/`. `graph_hash()` sorts the edges, formats the twelve static attributes of each to five decimals and feeds them to SHA-256. `weather_drift()` compares the stored snapshot with the new one and returns a list of human-readable reasons, which `get_or_build()` prints before rebuilding. The function returns the record, a flag saying whether it was rebuilt, and the reasons, so a caller can tell a reused front from a fresh one.

5.5 Module 3: Baselines

Each baseline returns a `BaselineResult` with the method name, its routes as (cost vector, path) pairs, the runtime and its settings. The A* variants call `networkx.astar_path` with a weight function that reads the chosen objective from the edge. `route_cost()` re-sums a path's cost on the graph, and `is_valid_route()` checks that the path starts and ends at the right nodes, that every edge exists and that no node repeats; every MOPSO particle is checked with it. The MOPSO archive is updated through the same `update_pareto_front()` helper that Tier 1 uses, so both methods apply exactly the same dominance rule.

5.6 Module 4: Tier 2 Selector

The weight bridge is a direct transcription of Equations (4.6) and (4.7):

```
def derive_weights(F: int, W: int, U: int) -> tuple:
    F = _validate_integer_tag("F", F, 1, 5)
    W = _validate_integer_tag("W", W, 1, 3)
```

```
    U = _validate_integer_tag("U", U, 1, 3)
    r_raw, d_raw, e_raw = F * 2, U * 3, W * 2
    total = r_raw + d_raw + e_raw
    return r_raw / total, d_raw / total, e_raw / total
```

Listing 5.2: Weight derivation (module4_tier2_payload.py)

`TierTwoSelector` computes the per-objective minima and maxima of the front once, in its constructor, so each selection only normalises and scores. A degenerate objective (all routes equal) normalises to zero instead of dividing by zero. `select_detailed()` validates the tags, derives and amplifies the weights, scores every route, filters by battery feasibility with `route_energy_wh()`, applies the weather gate, and builds a `SelectionResult`. The waypoints in the result carry node id, latitude, longitude, altitude above sea level and height above ground, taken from the graph. The record is written to `outputs/tier2/selection_<name>.json` and `.csv`. The simpler `select()` is kept for the demos and the sweep; it raises `WeatherNoGo` under a no-go but does not apply the battery filter, which is a known gap to be closed by routing all callers through `select_detailed()`.

5.7 Module 5: Evaluation

`module5_evaluation.py` contains the metric functions and the Tier 2 evidence functions. The exact hypervolume slices the normalised points along the first objective and recurses on the remaining two. `build_reference_front()` takes the routes of every method and keeps the non-dominated union. `tier2_evidence()` runs the four presets and the four competing pickers on one front, and `tier2_sweep()` runs all 225 combinations.

`run_benchmarks.py` builds the S1, S2 and S3 graphs from fixed seeds (101, 202, 303), runs NAMOA*, the A* baselines and MOPSO, and writes `benchmark_metrics.csv`, `tier2_evidence.csv` and bar charts of HV, IGD, spread, front size and runtime to `outputs/benchmarks/`. It takes `-scenario`, `-mopso-repetitions` (default 30), `-swarm-size` (100), `-iterations` (200), `-archive-size` and `-smoke`. `run_ablation.py` produces the ablation fronts, edge-risk maps and component maps in `outputs/ablation/`.

5.8 Module 6: Mission Export and Dashboard

`mission_export.py` represents each mission line as a `MissionItem` (index, current flag, frame, command, four parameters, latitude, longitude, altitude, auto-continue)

and serialises it as a tab-separated QGC WPL 110 line. Collinear points are merged when the bearing changes by at most 2° and the altitude slope by at most 0.5. A parser reads the file back, which the tests use to check that export and import agree. `dashboard.py` is a single Tkinter window with fields for the four tags, an embedded Matplotlib map and an export button.

5.9 Module 7: Simulation

The simulation side is run in WSL2 (Ubuntu 24.04) with Gazebo Harmonic, the ArduPilot Gazebo plugin and an ArduCopter SITL build. A delivery is simulated in five steps:

1. `prepare_sim_region.py -strict` freezes the region bundle from live OSM and DEM data; `-strict` refuses the hand-made test fixtures.
2. `gazebo_world_builder.py <region>` writes the world, meshes and the matching SITL command line.
3. `run_sim_mission.py` with `-preset`, `-payload`, `-all-presets` or `-baseline` plans the mission and writes `mission.plan`, `.waypoints` and `mission.json`. A tag-only change reuses the stored front and does not rerun NAMOA*.
4. The world is launched with the route markers, SITL is started with `-custom-location` at the depot, and `sitl_mission_runner.py` flies the mission.
5. `evaluate_gazebo_run.py` computes the flight metrics and plots.

The runner handles two ArduPilot version details. ArduCopter 4.7 and later name the waypoint speed parameter `WP_SPD` (m/s) instead of `WPNAV_SPEED` (cm/s), so the runner sets whichever one exists, and ArduPilot echoes `DO_CHANGE_SPEED` with frame 0 on read-back, which the comparison accepts.

5.10 Testing

Both code bases have offline unit-test suites that need no network and no API key. The planner suite has about 220 tests and the simulation suite 45. Table 5.4 lists what they cover.

Table 5.4: Test suites and what they verify

Test file	**What it checks**
`test_wind.py`	Headwind sign (wind from 90° is a headwind for an east-bound edge), crosswind, power-law shear, hourly row selection
`test_lattice.py`	Lattice size, 8-neighbour edges, east-bound edges cost more than west-bound under an east wind
`test_terrain.py`	DEM provider chain and cache, node altitude rule, every node within 120 m AGL
`test_risk.py`	Risk components in $[0, 1]$, NFZ node and edge removal, soft buffer, weather gate
`test_tier1_front.py`	NAMOA* front equals a brute-force front over all simple paths on a small graph; no route dominates another; every path valid
`test_store.py`	Reuse, expiry, graph-hash change, each drift threshold, identity check
`test_tier2_payload.py`, `test_tier2_hardening.py`	Weight formula, amplification, non-extreme pick for mixed cargo on a stored lattice front, monotonicity, validation errors, latency under 1 s, battery HOLD
`test_evaluation_metrics.py`	HV, IGD, spread on hand-computed cases; alignment score; pickers
`test_mission_export.py`	QGC WPL 110 round trip, collinear merge, frame 3 altitude relative to home
`test_ablation.py`, `test_overpass.py`, `test_dotenv.py`, `test_dashboard_helpers.py`	Ablation smoke run, Overpass retry and parsing, key loading, dashboard helpers
`Simulation/tests/*`	Bundle loading and frames, clearance checker, mission export (TAKEOFF → route → LAND), world builder geometry

6. Results and Discussion

This chapter reports the results the system currently produces. The numbers come from the first pass of the evaluation cycle described in Chapter 1.6, from a controlled wind experiment run for this report, from the ablation runner, and from the first SITL flights. They are preliminary in a specific sense: MOPSO was run at reduced settings, the risk weights and battery constants are not yet calibrated, and the synthetic density labels have a known defect. We state the caveats next to each result so that the reader

can judge how far it can be pushed.

6.1 Experimental Setup

Table 6.1: Scenarios used in this chapter

Scenario	**Size**	**Description**
S1 small sparse	50 nodes	Synthetic grid, seed 101, weather seeded
S2 medium	196–200 nodes	Synthetic grid, seed 202, weather seeded (benchmark) or set explicitly (wind experiment)
S3 large dense	500 nodes	Synthetic grid, seed 303, weather seeded
Thiruvanmiyur lattice	893 nodes, 6,752 edges	6 km box, 200 m spacing, offline fixture layers (ablation)
Flat lattice	225 nodes, 1,624 edges	3 km box, 200 m spacing, no layers (directional cost check)

All runs used the current code on a laptop-class CPU. In the first benchmark pass MOPSO used 24 particles, 30 iterations and 2 repetitions per scenario; the report-scale run (100, 200, 30) is the next step of the cycle.

6.2 Tier 1 Front Quality Against the Baselines

Table 6.2 gives the first-pass benchmark. Hypervolume (HV) and IGD are computed in normalised objective space against the union reference front of each scenario.

Table 6.2: First-pass benchmark on synthetic scenarios (HV higher is better; IGD and Δ lower are better)

Scenario	Method	Front	Time (s)	HV	IGD	Δ
S1 (50)	NAMOA* (Tier 1)	2	0.003	0.131	0.000	0.000
	A* distance	1	<0.001	0.121	0.866	–
	A* energy	1	<0.001	0.121	0.866	–
	A* risk	1	<0.001	0.011	0.866	–
	MOPSO	2	0.363	0.131	0.000	0.000
S2 (200)	NAMOA* (Tier 1)	16	0.162	1.2065	0.000	1.001
	A* distance	1	<0.001	0.107	1.021	–
	A* energy	1	<0.001	1.083	0.648	–
	A* risk	1	<0.001	0.011	0.887	–
	MOPSO	12	1.505	1.2061	0.017	1.027
S3 (500)	NAMOA* (Tier 1)	115	33.761	1.1235	0.000	0.666
	A* distance	1	0.001	0.062	0.861	–
	A* energy	1	0.002	0.674	0.442	–
	A* risk	1	0.002	0.013	1.116	–
	MOPSO	38	4.326	1.1146	0.043	0.814

NAMOA* has an IGD of zero in every scenario, which means its front contains every point of the reference front. That is what an exact method should give, and it confirms that the pruning rules do not throw away non-dominated routes. MOPSO recovers 12 of the 16 routes on S2 and 38 of the 115 on S3, and its HV is close to that of NAMOA* (a gap of 0.0004 on S2 and 0.009 on S3). The gap is small because MOPSO's archive still spans the extremes, even though it misses many interior points. Interior points are exactly the ones Tier 2 needs for mixed payloads, so the IGD and front-size columns matter more for this project than the HV column. NAMOA* also spreads its points more evenly (Δ of 0.666 against 0.814 on S3).

The single-objective A* routes reach high HV only when their one objective happens to dominate the volume, as A* energy does on S2. They cannot cover the front, and their IGD stays between 0.44 and 1.12.

Exactness has a cost. On S3, NAMOA* took 33.8 s against MOPSO's 4.3 s. Tier 1 runs once per region and weather window, not once per mission, so this is acceptable, but it is a target for the next cycle (front thinning, a tighter heuristic and a coarser lattice in open areas). Figures 6.1 and 6.2 plot the same data.

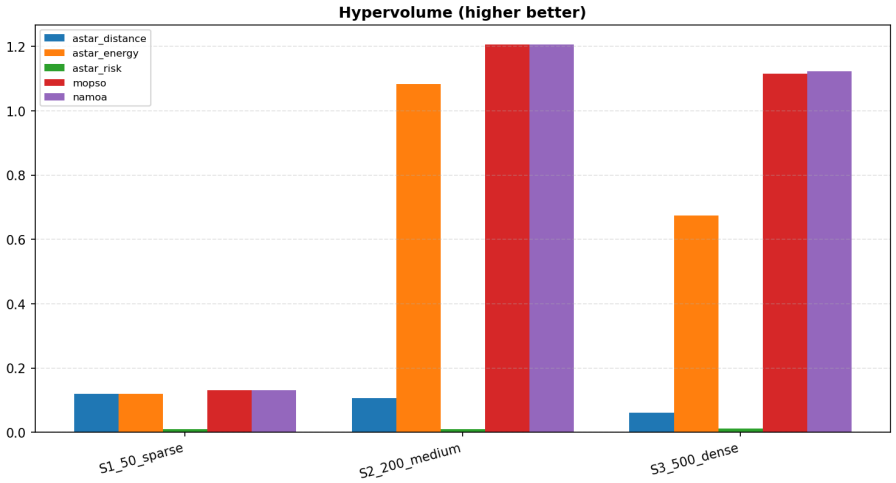


Figure 6.1: Hypervolume per method and scenario (first pass)

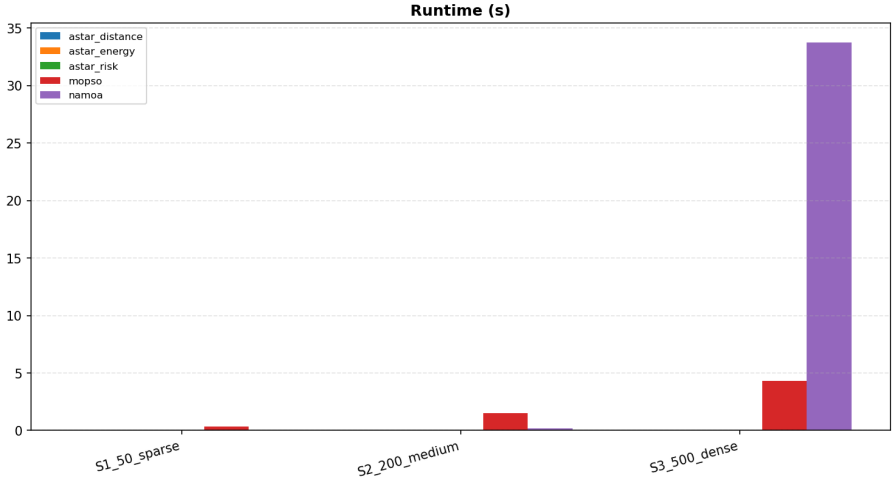


Figure 6.2: Runtime per method and scenario (first pass)

6.3 Effect of Wind Speed and Direction

This experiment isolates the wind model. The S2 synthetic graph (196 nodes, 1,404 edges, seed 202) was costed under calm air and under a 25 km/h wind with 35 km/h gusts blowing from each of eight compass directions. Nothing else changed: the same graph, the same static layers, the same source and target. Table 6.3 shows the result.

Table 6.3: Effect of wind direction on the S2 front (25 km/h wind, 35 km/h gust)

Wind from	**Front**	**Labels**	**Min ε**	**Min ρ**	**Fragile pick** (d, ε, ρ)	**Distinct**
Calm	36	869	24.41	0.07	(26.78, 26.92, 0.11)	4
0° (N)	14	1019	31.01	3.24	(26.16, 32.82, 3.40)	2
45° (NE)	6	545	31.34	1.89	(26.16, 33.63, 1.89)	2
90° (E)	11	876	30.94	3.31	(26.16, 32.75, 3.45)	2
135° (SE)	14	1100	27.13	4.27	(26.16, 29.59, 4.42)	2
180° (S)	14	989	26.18	3.24	(26.16, 27.99, 3.40)	2
225° (SW)	6	482	24.58	1.89	(26.16, 26.87, 1.89)	2
270° (W)	11	801	26.22	3.31	(26.16, 28.03, 3.45)	2
315° (NW)	14	1084	27.21	4.27	(26.16, 29.67, 4.42)	2

The first observation is that direction alone matters a great deal. At the same speed, the cheapest route on the front needs 24.58 energy units with the wind from the south-west and 31.34 with it from the north-east, a difference of 27.5%. A planner that used wind speed without direction could not see this.

The risk and energy objectives also respond to different properties of the wind. Opposite directions (0° and 180°, or 45° and 225°) give the same minimum risk, because crosswind depends only on the axis of the wind. They give very different minimum energy, because the headwind penalty depends on which way along that axis the drone is flying. This is the behaviour Equation (4.4) predicts, and it is why both components are needed.

Finally, wind changes the shape of the front as well as its position. The front shrinks from 36 routes in calm air to between 6 and 14, and the four presets land on only two distinct routes. Under a steady wind, edges with strong crosswind are both costlier and riskier, so energy and risk become correlated and there are fewer real trade-offs left. The fragile medical pick stays the lowest-risk option on every front, but its energy varies from 26.87 to 33.63 with direction. Whether the collapse to two picks is the right behaviour or an artefact of min-max normalisation on a short front is one of the questions the next evaluation pass has to settle. Figure 6.3 repeats the experiment in 15° steps.

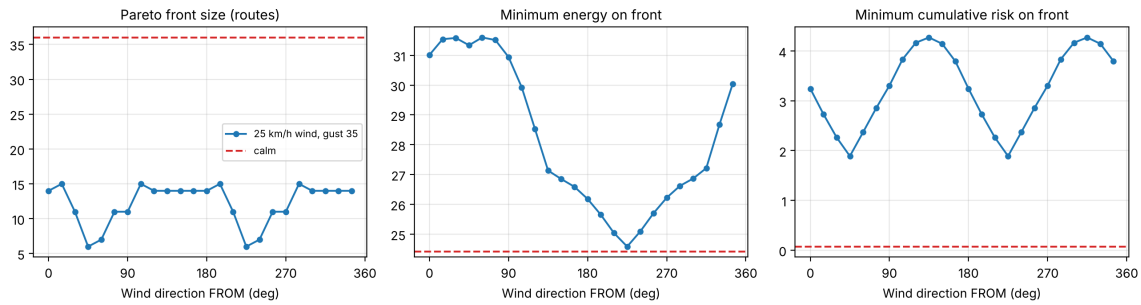


Figure 6.3: Front size, minimum energy and minimum risk on S2 as a 25 km/h wind turns through 360°; the dashed line is calm air

The same effect shows up edge by edge. Table 6.4 lists the mean energy per kilometre and the mean edge risk score by heading on a flat 3 km lattice with no other layers, under a 25 km/h wind from the east.

Table 6.4: Directional edge costs on a flat lattice, 25 km/h wind from 90°

Edge heading	0°	45°	90°	135°	180°	225°	270°	315°
ε/d	1.104	1.270	1.278	1.270	1.104	1.074	1.000	1.074
ρ_{edge}	0.225	0.170	0.038	0.170	0.225	0.170	0.038	0.170

East-bound edges fly into the wind and cost 27.8% more energy per kilometre than west-bound edges, which fly with it and cost exactly their distance because tailwind earns no credit. North- and south-bound edges take the full crosswind and carry six times the risk score of east–west edges, whose remaining 0.038 comes only from the gust term. On a flat lattice with no obstacles the front still has a single route, since the straight diagonal is best on every objective. Trade-offs need the wind to interact with terrain, buildings or restricted zones, which is what the next two results show.

6.4 Contribution of Each Risk Layer

The ablation runner rebuilt the Thiruvanmiyur lattice front with all layers and with wind, terrain or buildings removed, under a 30 km/h wind from the south-west. These runs used the offline fixture layers, which are hand-made test data shaped like the real region, so the numbers show how each layer acts on the planner rather than real-world values. The live-data run is part of the next cycle.

Table 6.5: Ablation on the 6 km Thiruvanmiyur lattice (fixture layers, 30 km/h wind from 225°)

Variant	**Front**	**Time (s)**	**Min ε**	**Min ρ**	**Max ρ**	**Mean ρ_{edge}**
All layers	196	11.35	8.82	8.85	13.12	0.199
No wind	205	5.38	7.84	0.10	4.24	0.061
No terrain	56	1.36	8.81	8.81	13.08	0.198
No buildings	190	8.32	8.82	8.85	12.26	0.192

Wind is the largest single driver of risk in this setting. Removing it lowers the mean edge risk from 0.199 to 0.061 and the risk of the safest route from 8.85 to 0.10, and it lowers the minimum energy by 11%. Terrain mostly drives the size of the front: without grades there are far fewer distinct trade-offs (56 routes against 196), because altitude changes are what separate otherwise similar paths. Buildings raise the upper end of the risk range. Figure 6.4 shows the edge-risk maps for the four variants; the hole near the top centre is a restricted zone whose nodes and crossing edges were removed.

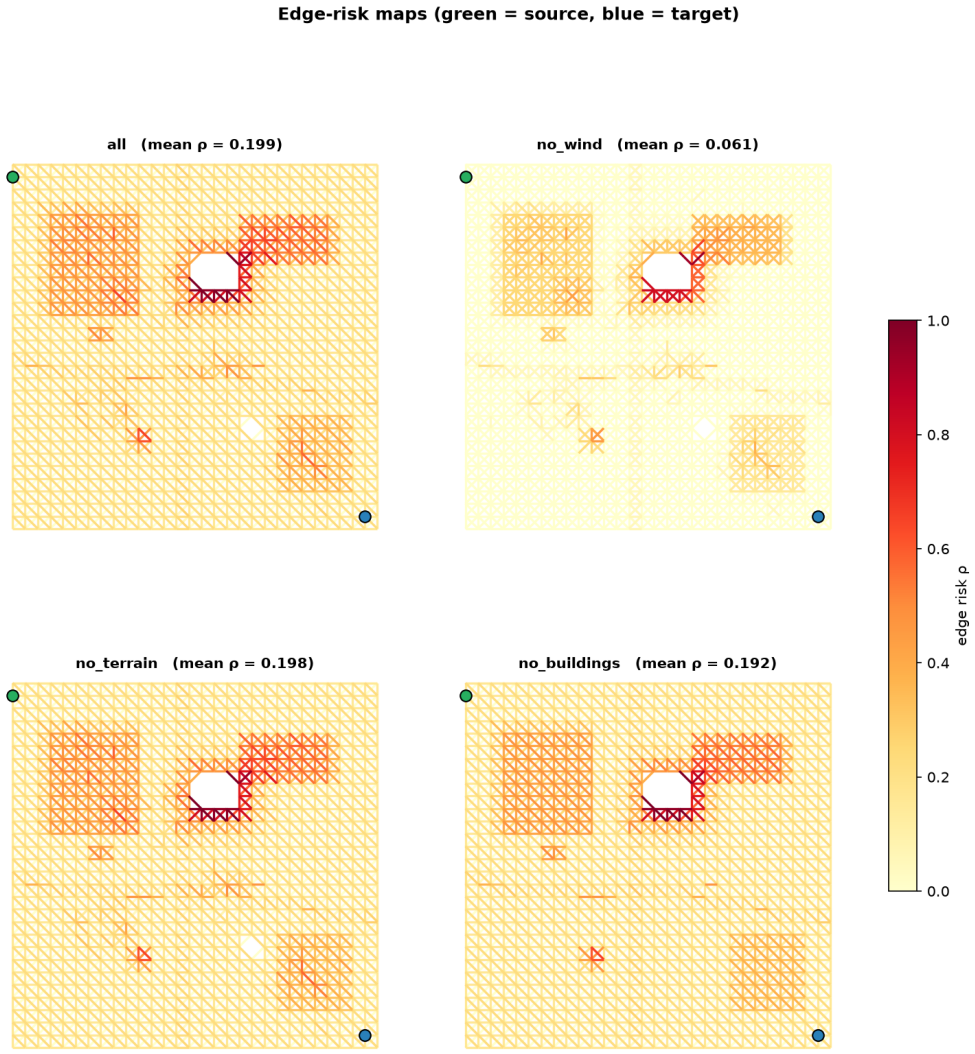


Figure 6.4: Edge risk maps for the ablation variants (fixture layers)

6.5 Tier 2 Selection

Table 6.6 shows the Tier 2 picks on the S2 and S3 fronts of the benchmark. The ranks are the selected route's position on each objective within the front (1 is best).

Table 6.6: Tier 2 picks on the S2 (16 routes) and S3 (115 routes) fronts; last column is latency in ms

Front	Preset	Rank d	Rank ε	Rank ρ	Point type	ms
S2	Fragile medical	7	5	10	non-extreme	0.030
	Emergency medicine	3	1	14	min-energy	0.025
	Disaster relief	3	1	14	min-energy	0.022
	Mixed cargo	4	2	13	non-extreme	0.023
S3	Fragile medical	91	81	25	non-extreme	0.144
	Emergency medicine	22	26	94	non-extreme	0.140
	Disaster relief	28	1	88	min-energy	0.136
	Mixed cargo	84	59	32	non-extreme	0.136

On the 115-route S3 front, three of the four presets select non-extreme points, and the four presets select four different routes. Disaster relief takes the minimum-energy route, which matches its dominant weight. The emergency medicine pick is worth a closer look. It is the 22nd-shortest route, yet it is only 1.9% longer than the shortest route on the front. Its weights are dominated by distance, but not completely, so the utility accepts a very small distance penalty in exchange for a lower risk. A single-objective planner cannot make that trade, and a fixed-weight planner would make the same trade for every payload. The fragile medical pick is 25th of 115 in risk rather than first, which again reflects its non-zero distance and energy weights. Every selection took well under a millisecond, far inside the one-second target.

On the smaller S2 front the picks are less spread: emergency medicine and disaster relief share the minimum-energy route, which on this graph is also near the shortest. This is the same pattern as in the wind experiment, and it points to the size of the front as the factor that limits how many distinct choices Tier 2 can make.

Figure 6.5 shows the four presets on a 2 km lattice over Thiruvanmiyur with OpenStreetMap roads drawn underneath for reference. The fragile medical and mixed cargo presets take a slightly longer path (2.6 km) with a cumulative risk of 1.57, while emergency medicine and disaster relief take 2.5 km paths with risks of 1.88 and 1.89.

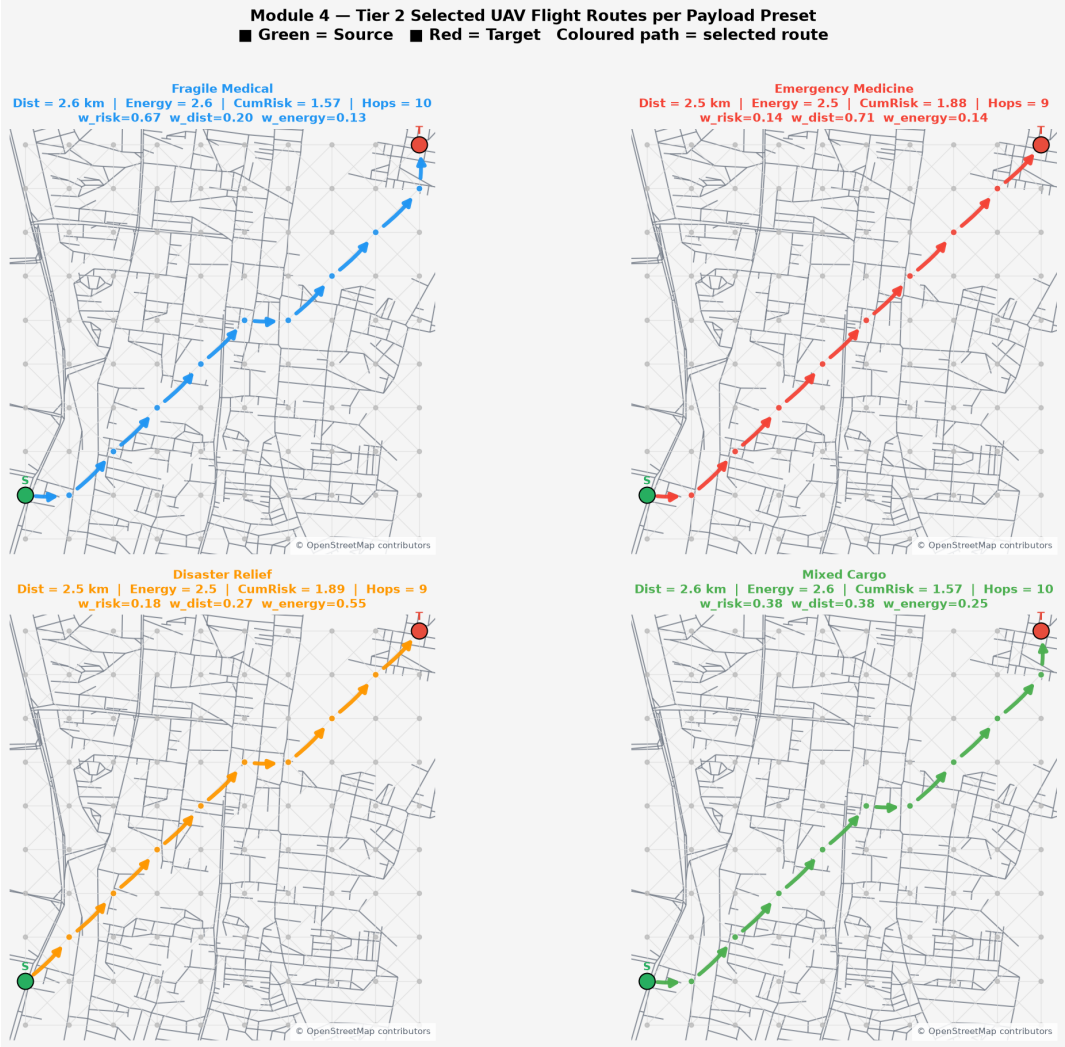


Figure 6.5: Tier 2 routes for the four presets on a 2 km Thiruvanmiyur lattice

6.6 Simulation Results So Far

The SITL mission runner has been flown against ArduCopter SITL (4.8.0-dev, built-in quadcopter physics, run at five times real time) with missions produced by the planner from a fixture bundle. Mission upload, read-back, the switch to AUTO, landing and disarm all succeeded. The cross-track root-mean-square error between the flown and planned tracks was 0.11–0.13 m, and the landing offset from the destination pad was 0.01–0.02 m. These numbers show that the mission chain from Tier 2 to the autopilot is correct, including the altitude frame. They do not yet say anything about the city model, wind in the simulator or the battery, which are the open items listed in Chapter 1.6.

6.7 Comparison With Existing Work

Table 6.7 compares the system with the three works reviewed in Section 1.3.

Table 6.7: Comparison with the reviewed works

Property	**Chokkalingam & Ganesan [2]**	**Tsoutsouras et al. [3]**	**Ren et al. [10]**	**This work**
Front construction	POPSO, stochastic	None	CDNSGA-II, stochastic	NAMOA*, exact and deterministic
Objectives	Multiple, scalarised for the final pick	Energy and speed	Distance, safety	Distance, energy, risk
Final route selection	Weighted sum, weights tuned per mission	Not applicable	Crowding distance	Utility from payload tags, argmax over the full front
Payload in selection	No	Mass in the energy model only	No	Fragility, weight class, urgency, priority
Wind direction in edge costs	Not reported	Not reported	Not reported	Headwind and crosswind per edge, gust, shear, weather gate
Front reuse	Re-run per mission	Not applicable	Re-run per mission	Stored front with drift-based refresh

The comparison is qualitative for now. Their datasets and code are not public, so the quantitative comparison uses the selection rules they describe (crowding distance as in Ren et al., fixed weights as in Chokkalingam and Ganesan) applied to our fronts, through the payload-alignment score of Section 4.2.5. That comparison runs at report scale in the next pass.

6.8 Discussion and Next Steps

So far the results support a few claims. The exact Tier 1 front covers the reference front completely while MOPSO recovers a third of it on the largest scenario. Tier 2 picks non-extreme routes for mixed payloads in well under a millisecond, and on a large front different payloads get different routes. The directional wind model changes energy, risk and the shape of the front in the way the physics predicts, and a direction-blind model would miss a 27.5% energy difference at a fixed wind speed.

They also expose what the next passes must fix. The MOPSO comparison needs the report-scale settings and per-repetition statistics. The density labels must reach the graph builder. The monotonicity check must be grouped by scenario. The collapse of the front under steady wind needs a study of normalisation and possibly of the weight multipliers. The ablation and the Tier 2 results must be repeated on live OpenStreetMap data, and the risk weights and battery constants must be calibrated. On the simulation side, the Gazebo world, wind in the simulator and repeated flights per preset come next. Each of these feeds the cycle described in Chapter 1.6.

References

[1] Jian Deng, Honghai Zhang, Dikun Feng, Zihan Yu, and Zeyu Liu. Multi-objective UAV trajectory planning algorithm based on the hybrid MS-INSGA2 strategy. *Journal of King Saud University – Computer and Information Sciences*, 38:562, 2026. doi: 10.1007/s44443-026-00930-0.

[2] K. Chokkalingam and R. Ganesan. Dynamic parameter adaptation in Pareto-optimal particle swarm optimization for efficient UAV path planning. *RAIRO – Operations Research*, 60(1):281–305, 2026. doi: 10.1051/ro/2025159.

[3] Vasileios Tsoutsouras, Joseph Story, and Phillip Stanley-Marbell. Payload-mass-aware trajectory planning on multi-user autonomous unmanned aerial vehicles, 2020. doi: 10.48550/arXiv.2001.02531.

[4] Avionics International. Drone delivery crash in Switzerland raises safety concerns as UPS forms subsidiary. Avionics International, August 2019. `https://www.aviationtoday.com/2019/08/08/drone-delivery-crash-in-switzerland-raises-safety-concerns/`.

[5] J. Sherwood et al. Quantifying the effects of vibration on medicines in transit caused by fixed-wing and multi-copter drones. *Drones*, 5(1):22, 2021. doi: 10.3390/drones5010022.

[6] N. Scott et al. Stability of medicines transported by cargo drones: Investigating the effects of vibration from multi-stage flight. *Drones*, 7(11):658, 2023. doi: 10.3390/drones7110658.

[7] R. Mathew et al. Preanalytic integrity of blood samples in uncrewed aerial vehicle (UAV) medical transport: A comparative study. *Drones*, 8(9):517, 2024. doi: 10.3390/drones8090517.

[8] IEEE Spectrum. In the air with Zipline's medical delivery drones. IEEE Spectrum, 2019. `https://spectrum.ieee.org/in-the-air-with-ziplines-medical-delivery-drones`.

[9] K. Deb, S. Agrawal, A. Pratap, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.

[10] Q. Ren, Y. Yao, G. Yang, and X. Zhou. Multi-objective path planning for UAV in the urban environment based on CDNSGA-II. In *Proceedings of the IEEE*

International Symposium on Service-Oriented System Engineering (SOSE), 2019. doi: 10.1109/SOSE.2019.00059.

[11] Open-Meteo. Free weather API and elevation API. `https://open-meteo.com`, 2026. Accessed October 2026.

[12] Ministry of Civil Aviation, Government of India. The drone rules, 2021. Gazette of India, G.S.R. 589(E), August 2021.

[13] Joint Authorities for Rulemaking of Unmanned Systems (JARUS). JARUS guidelines on specific operations risk assessment (SORA). JAR-DEL-WG6-D.04, Edition 2.0, 2019.

[14] Erik W. Peterson and Joseph P. Hennessey. On the use of power laws for estimates of wind power potential. *Journal of Applied Meteorology*, 17(3):390–394, 1978.

[15] Lawrence Mandow and José Luis Pérez de la Cruz. Multiobjective A* search with consistent heuristics. *Journal of the ACM*, 57(5):27:1–27:25, 2010. doi: 10.1145/1754399.1754400.

[16] Carlos A. Coello Coello, Gregorio Toscano Pulido, and Maximino Salazar Lechuga. Handling multiple objectives with particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, 8(3):256–279, 2004. doi: 10.1109/TEVC.2004.826067.

[17] ArduPilot Development Team. ArduPilot software in the loop (SITL) simulator. `https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html`, 2026. Accessed October 2026.

[18] Open Robotics. Gazebo harmonic. `https://gazebosim.org`, 2023. Accessed October 2026.

[19] Eckart Zitzler and Lothar Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271, 1999. doi: 10.1109/4235.797969.

[20] Aric A. Hagberg, Pieter J. Swart, and Daniel A. Chult. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15, 2008. doi: 10.25080/TCWV9851.

[21] Geoff Boeing. OSMnx: New methods for acquiring, constructing, analysing, and visualising complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017. doi: 10.1016/j.compenvurbsys.2017.05.004.

[22] Lorenz Meier et al. MAVLink: Micro air vehicle communication protocol. `https://mavlink.io`, 2013.